COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Graphics and Game-Engine Construction

Student handbook / expanded Volume 05

20 lessons · Modules 21-25 · Worked examples · 100 lesson questions · Gate E

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026

Self-study: derive, implement, test and explain.

Contents

How to use this volume 3
Module 21 — From geometry to a software renderer 5
21.1 — Images, color and coordinate conventions 6
21.2 — Projection and clipping before perspective division 8
21.3 — Triangle coverage, depth and perspective-correct interpolation 10
21.4 — Rays, intersections and acceleration structures 12
Module 21 review and project handoff 14
Module 22 — Explicit GPU programming and graphics APIs 15
22.1 — GPU work submission, resources and pipeline state 16
22.2 — Buffers, shaders, layouts and resource binding 18
22.3 — Synchronization, hazards and frames in flight 20
22.4 — Presentation, resizing and device capability policy 22
Module 22 review and project handoff 24
Module 23 — Lighting, materials and real-time image quality 25
23.1 — Radiometry, reflectance and physically based materials 26
23.2 — Textures, mipmaps, filtering and tangent-space detail 28
23.3 — Shadows, light selection and rendering-path tradeoffs 30
23.4 — HDR, tone mapping, temporal reconstruction and validation 32
Module 23 review and project handoff 34
Module 24 — Engine architecture, simulation and resource ownership 35
24.1 — Fixed updates, interpolation and overload policy 36
24.2 — Entities, components, handles and data-oriented storage 38
24.3 — Allocators, task graphs and frame-resource lifetime 40
24.4 — Subsystem boundaries, save formats and reloadability 42
Module 24 review and project handoff 44
Module 25 — Asset pipelines, editors and an integrated engine 45
25.1 — Importing, cooking and content-addressed asset builds 46
25.2 — Streaming, residency and memory budgets 48
25.3 — Editors, inspection and undoable commands 50
25.4 — Engine integration and reproducible demonstration 52
Module 25 review and project handoff 54
Gate E specification 55
Sources and provenance 56

Graphics and Game-Engine Construction

How to use the expanded volume

Build explanations and evidence, not only pictures. This volume moves from an image buffer to a CPU visibility reference, then to asynchronous GPU work, materials, engine ownership and a reproducible small-engine demonstration. Existing game engines may be used for comparison, but the central implementation is your own.

Package numbering differs from volume numbering: Package 06 expands Volume 05. It follows the systems/software-engineering volume, preserves Modules 21-25 and ends at Gate E. Later Volume 06 adds gameplay production systems; later graduate volumes deepen graphics/engine specialization.

Read each lesson, predict the worked result, attempt the original assignment using the expanded workbook procedure, complete its coding exercise and answer all five questions before opening the key. Questions 1-3 are preserved; 4-5 are new application checks. The 25 module-review questions are additional. A correct reference-program run is a useful baseline, not credit for implementing your own solution.

Scope and route

Module	Focus	Main evidence
21	Geometry to software rendering	Byte-tested images, clipping/coverage/depth tests, ray/BVH oracle
22	Explicit GPU programming	Capability policy, byte layouts, submission/retirement evidence
23	Lighting and image quality	Material assumptions, filtering, lighting budgets, temporal tests
24	Engine architecture	Fixed-step world, stable identity, dependency graph, safe persistence
25	Assets and integration	Deterministic cooker, residency/inspection, clean reproducible engine

The original allowance is **420-650 planning hours** for this volume, including implementation, assigned study, projects, review and remediation. It is not the time needed to read these pages, a deadline, or an academic-credit equivalence. The five module projects combine into Gate E rather than adding a sixth independent capstone.

Prerequisite gate

Demonstrate vectors/transforms and homogeneous coordinates; barycentric/algebraic reasoning; sampling and basic probability; C++ memory/lifetime and testing; resource ownership, concurrency and build discipline. Use the previous mathematics and systems checkpoints rather than guessing from engine-editor familiarity. The module-specific prerequisites remain listed below.

An optional eight-question readiness check is in START_NEXT.pdf; its answers are separate in ANSWER_KEY.pdf and do not count toward the 100 lesson questions. Weak prerequisites should trigger a focused repair task, not silent reliance on reference code.

A single baseline contract

The CPU reference uses a right-handed camera looking down -Z, column-vector transformations, clip depth 0 <= z <= w, screen origin upper-left, centers at (x+0.5,y+0.5), 1/256-pixel quantized triangle coverage and strict-less depth against a clear value of 1.0. It writes an explicitly labeled sRGB-coded RGB8 PPM variant. The exact far plane can pass clipping but fail the strict-less depth comparison; this is a documented baseline policy, not a missing test.

The checker is point sampled and the material is unlit vertex color. The source is an oracle for selected geometry/ownership contracts, not a photorealistic engine. Native GPU APIs can differ in sample precision and conventions. Reconcile them before claiming image equivalence. [S01, S02]

Supplied versus assigned

Supplied: expanded teaching, procedures, 100 lesson answers, 25 review answers, 20 small starter/reference contract pairs, a C++ CPU renderer, sphere BVH/brute-force comparison, a deterministic mesh/image cooker, source-only WebGPU compute-to-draw bridge, evidence templates and local CPU validation.

Assigned: your full lesson labs, native or selected GPU backend project, microfacet material implementation, shadow/lighting comparison, temporal method, engine world/resource design, asynchronous residency, editor/inspector with undo/save/load, and the integrated Gate E engine. Small helper functions do not silently fulfill their parent engineering systems.

GPU hardware execution, macOS/Windows execution and the required independent Gate E reviewer run were **not performed during packaging**. validation/TESTED_ENVIRONMENT.json distinguishes actual runs from supported design intentions. A software-only progression is acceptable while GPU access is unavailable; do not label pending GPU work as passed.

Module 21 — From geometry to a software renderer

Prerequisites: Modules 07-08, 11-16; complete the mathematics checkpoint.

Planning allocation: 90-140 planning hours, including the project's work and review.

Lesson	Outcome
21.1 — Images, color and coordinate conventions	Define the numerical and geometric meaning of an image buffer.
21.2 — Projection and clipping before perspective division	Transform geometry without confusing visibility with division by depth.
21.3 — Triangle coverage, depth and perspective-correct interpolation	Produce crack-resistant filled triangles with correct varying attributes.
21.4 — Rays, intersections and acceleration structures	Build a second visibility method and compare its costs with rasterization.

Original module project

Build a CPU renderer that writes a portable image file. It must clip geometry, rasterize triangles, resolve visibility and interpolate attributes correctly. Keep it as a reference implementation for future GPU work.

Integrating milestones

1. P21-A: image contract and byte-checked PPM output.
2. P21-B: six-plane homogeneous clipping with attributed vertices.
3. P21-C: quantized coverage, depth and perspective-correct UVs with comparison images.
4. P21-D: a sphere or triangle ray path and BVH/brute-force comparison.

Evidence to retain

- Coordinate contract; unit tests; clipped polygon dumps; coverage-count proof.
- Affine/corrected images; near-plane sequence; BVH seed and raw build/query measurements.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 21.1 / MODULE 21

Images, color and coordinate conventions

> **Learning outcome:** Define the numerical and geometric meaning of an image buffer.

An image is a contract, not just an array

A pixel address is meaningful only after you specify the dimensions, row stride, bytes per pixel, channel order, origin and transfer function. For interleaved RGB8, the address of channel c at integer coordinate (x,y) is base + y*stride* + 3x + c. A 641-pixel row may occupy more than 1,923 bytes when an interface requires alignment. Padding is storage, not additional visible pixels. Use checked multiplication before allocating width*height*channels; a wrapped size can turn an apparently valid image into an out-of-bounds write.

Keep three coordinate systems separate in your notebook: integer image indices, continuous sample positions and geometric coordinates. In this package, memory rows run top to bottom, x increases right, and the sample for pixel (x,y) is at (x+0.5,y+0.5). Normalized device coordinates use x right and y up. Consequently, viewport mapping flips y. Treat that conversion as an explicit boundary, not an unexplained negative sign scattered through shaders.

Light values and display values

A stored byte does not say whether it represents linear light, nonlinear color, a normal component, an object ID or roughness. Lighting arithmetic in this volume uses a linear RGB working space with fixed primaries. sRGB-encoded color inputs are decoded before lighting; output is encoded once after exposure and tone mapping. Color primaries and a transfer function are different parts of a color-space definition. Merely applying a power function is not a complete color-management system.

For normalized sRGB input s, decode with s/12.92 when s <= 0.04045, otherwise ((s+0.055)/1.055)^2.4. For linear output l in [0,1], encode with 12.92*l when l <= 0.0031308*, *otherwise 1.055*l^(1/2.4)-0.055. These piecewise equations are not interchangeable with a universal gamma of 2.2. Quantize only at the output boundary, after handling invalid or out-of-range values deliberately. [S03]

Data textures and portable output

A normal-map channel is numerical data. Applying sRGB decoding changes the vector; it is not a cosmetic adjustment. Depth, roughness, masks and object IDs likewise need semantic rules. A normal map may require remapping [0,1] to [-1,1] and normalization, while an ID texture must not be linearly blended between unrelated IDs.

The P6 PPM format is useful because its header and raster can be inspected without a graphics SDK. This package writes three 8-bit RGB channels, top row first. Its preview files intentionally use the common sRGB-coded PPM variant, documented in a header comment; strict PPM color semantics are based on BT.709, so the files are not color-managed reference masters. Preserve linear numerical test data separately and never judge transfer-function correctness solely by an uncalibrated viewer. [S02]

Engineer the boundary first

Begin with four asymmetric corner markers and a gradient. Symmetric images hide flips, swaps and incorrect origins. Test bytes directly before looking at the picture. This establishes a trustworthy output surface for the clipping and visibility work that follows.

Worked example

The original packed-image example is 320*180*4 = 230,400 bytes for RGBA8, excluding metadata. Exporting that same size as RGB8 PPM uses 172,800 raster bytes, plus the ASCII header. Those are different representations, not conflicting answers.

For a 3-by-2 RGB8 image with a 12-byte stride, pixel (2,1) begins at *1*12 + *2*3 = byte 18. The allocation has 24 bytes, although only 18 are visible pixel data. A tightly packed exporter copies nine useful bytes per row rather than copying padding into the raster.

For equal-area black and white samples, a linear-light average is 0.5. Encoding that result gives about 0.7354, or byte 188 after nearest-integer quantization. Averaging the encoded bytes directly gives about 128 instead. The two paths visibly disagree because encoding is nonlinear. Keep this two-sample experiment as a regression test.

Original lab assignment

Write a PPM image containing a gradient and coordinate markers. Pass: the dimensions, orientation and channel values are verified by tests, and a README states all coordinate and color assumptions.

Assigned study

R19: image representation and transformations; R23: Radiometry, Spectra, and Color.

Focused study: S02: PPM header, raster order and nonlinear variants. S03: sRGB texture semantics; R23: color representations. Derive the address expression on paper.

Check your understanding

1. Why must image stride be recorded?
2. How many bytes are in the tightly packed example image?
3. Should a normal map automatically receive sRGB decoding?
4. A 3-by-2 RGB8 image has stride 12. What is the byte offset of pixel (2,1), and how large is its allocation?
5. Why does averaging encoded black and white produce a different result from encoding their linear-light average?

LESSON 21.2 / MODULE 21

Projection and clipping before perspective division

> **Learning outcome:** Transform geometry without confusing visibility with division by depth.

Projection must preserve the information needed for clipping

A camera transform maps world coordinates into a view frame. Projection produces a homogeneous point (xc,yc,zc,wc), not a final screen position. Perspective division comes later: x_ndc=xc/wc and similarly for y and z. Dividing a point close to wc=0 can create enormous or nonfinite coordinates. Worse, vertices behind the camera can change apparent sides after division. Visibility is a geometric operation, not a test that a quotient happened to fit on the screen.

Use one consistent clip volume. The reference path uses -w <= x <= w, -w <= y <= w and 0 <= z <= w. This is a zero-to-one depth convention; do not import an OpenGL-style near inequality z >= -w without changing the projection and tests. The chosen camera looks along -Z, uses column vectors and radians, and maps the near plane to z_ndc=0 and the far plane to 1. A backend adapter owns any further convention changes. [R19, S01]

Half-spaces make clipping systematic

Represent each plane by a signed function f(p), with f(p)>=0 meaning inside. The six functions are x+w, w-x, y+w, w-y, z and w-z. Traverse a polygon edge from a to b. When the endpoints lie on different sides, the intersection parameter is t=f(a)/(f(a)-f(b)). The new vertex is a+t*(b-a). Interpolate the original per-vertex attributes with that same t while they still belong to the undivided geometric edge.

The four edge cases are inside-to-inside, inside-to-outside, outside-to-inside and outside-to-outside. Their output is respectively b; intersection; intersection then b; and nothing. Applying this rule around the polygon is Sutherland-Hodgman clipping against one convex half-space. Repeat for each plane, then triangulate a surviving polygon as a fan. A triangle may become a quadrilateral or a larger polygon; the output need not contain exactly three vertices.

Boundaries and numerical policy

Declare points exactly on a plane to be inside. Shared edges must use the same plane equations and interpolation convention. Arbitrary per-triangle epsilon changes can produce inconsistent intersections. The small reference uses double precision and explicit finite-input checks, but is not an exact-predicate geometry library. Record its supported coordinate range and tolerance; add stronger robustness before treating it as a universal oracle.

Clipping does not justify dividing by zero. After all planes, reject singular vertices whose w does not meet the documented positive threshold. Degenerate polygons with zero projected area should produce no fragments. Avoid clamping a negative w to a positive constant: that invents geometry instead of clipping it.

A practical debugging sequence

Draw an unprojected triangle first, then a projected triangle fully inside the frustum, then one crossing only the near plane. Only after these pass should you test all six planes together. Preserve polygon vertex dumps before and after clipping. A blank result is easier to explain with plane distances than with an image alone.

Worked example

For the near half-space f(p)=z, let a=(0,0,-1,1) and b=(2,0,1,2). Then f(a)=-1 and f(b)=1, so t=-1/(-1-1)=0.5. The intersection is (1,0,0,1.5), with x_ndc=2/3 after division. Clipping after first dividing a and b would interpolate a different projected segment parameter and would lose the correct attribute relationship.

For the original normalization question, x=2 and w=4 gives x_ndc=0.5. That one quotient says nothing about the other five clip inequalities.

With a 90-degree vertical field of view, aspect 1, near 0.1 and far 100, use w=-z_view and z_clip=A*z_view+B*, *where A=far/(near-far) and B=far*near/(near-far). Substituting z_view=-0.1 yields z_ndc=0; substituting -100 yields 1. Test those endpoints before rendering any scene.

Original lab assignment

Implement clipping against one plane, then the complete chosen clip volume. Pass: triangles crossing the near plane remain finite and correctly shaped, and tests cover points exactly on the boundary.

Assigned study

R19: perspective transformations, clipping and the graphics pipeline; R23: Cameras.

Focused study: R19: homogeneous projection and clipping. S01: clip-space conventions for the chosen shader route. Verify the chosen near/far mapping algebraically.

Check your understanding

1. Why clip before perspective division?
2. What normalized component follows from x=2 and w=4?
3. Is deleting every outside vertex a valid triangle-clipping algorithm?
4. For plane f=z, endpoints have distances -2 and 6. What interpolation parameter reaches the boundary?
5. In a zero-to-one clip volume, does z=-0.2,w=1 pass the near plane? What changes under a minus-one-to-one depth convention?

LESSON 21.3 / MODULE 21

Triangle coverage, depth and perspective-correct interpolation

> **Learning outcome:** Produce crack-resistant filled triangles with correct varying attributes.

Coverage is a discrete geometric decision

After clipping and division, map the triangle to pixel space. A signed edge function E(a,b,p)=(bx-ax)(*py-ay)-(by-ay)*(px-ax) tests which side of the directed edge contains p. Orient the projected vertices so the signed area is positive in the chosen algebraic coordinates. Evaluate at pixel centers, not at integer corners. Divide the three opposite-edge values by the signed area to obtain barycentric coordinates lambda0, lambda1 and lambda2.

A mathematical point on a shared edge belongs to both closed triangles. A framebuffer cannot allow that ambiguity when blending, counting fragments or assigning IDs. Use a half-open fill convention. For this package's y-down image coordinates and positive-area edge function, include a zero-valued edge when dy<0, or dy==0 and dx>0. A reversed shared edge then belongs to exactly one triangle. The reference quantizes screen positions to a 1/256-pixel grid before integer coverage tests. Quantization is a documented rasterization choice, not exact continuous geometry. [R19]

Visibility is separate from color

Initialize depth to the far value, here 1, and accept a fragment when its z_ndc is strictly less. This tie rule means an exactly far-plane fragment does not replace a cleared depth of 1. Document that boundary. Draw opaque overlapping triangles in both orders and compare the result. Triangle sorting alone is not sufficient for intersecting geometry; depth testing resolves visibility at each sample.

For a conventional perspective projection, z_ndc at the sample is the screen-barycentric affine interpolation of vertex zc/wc. Do not apply a second perspective correction to this already projected depth. A useful code review labels every quantity by its space: view z, clip z, reciprocal w or normalized depth. Reusing the name z for all four hides mistakes.

Why texture coordinates need a ratio

A UV coordinate is affine along the original surface, not generally affine after perspective projection. For an attribute a, compute q=sum(lambda_i/w_i), numerator=sum(lambda_i*a_i/w_i), and a=numerator/q. You interpolate a/w and 1/w, then divide. Colors and other smooth surface attributes follow the same rule when they represent quantities attached to that surface. Flat attributes, such as material IDs, need a separate flat/provoking-vertex policy.

The ratio reduces to affine interpolation when all vertex w values are equal. More generally, a constant attribute is unchanged either way, but that does not make the methods equivalent for arbitrary attributes. A receding checkerboard reveals the difference far more clearly than a solid triangle.

Robustness and reference comparisons

Clamp the integer bounding box to the target before touching memory. Reject zero-area triangles and nonfinite intermediate values. Test both windings, subpixel translation and triangles partly outside each screen edge. A screenshot without cracks is weak evidence: add an integer coverage-count image and check that a two-triangle rectangle has exactly one owner per intended sample. When comparing to hardware,

account for its subpixel precision, edge conventions and sample positions; this CPU path is an educational reference, not a promise of bit-identical GPU rasterization.

Worked example

Take barycentric weights (0.2,0.3,0.5), vertex w values (1,2,4), and scalar u values (0,1,1). The affine result is 0.8. The perspective numerator is 0 + 0.3/2 + 0.5/4 = 0.275. The reciprocal-w denominator is 0.2 + 0.3/2 + 0.5/4 = 0.475. Therefore u=0.275/0.475=11/19, approximately 0.578947.

If those vertices have z_ndc values (0.1,0.4,0.8), the sample depth is 0.2*0.1 + 0.3*0.4 + 0.5*0.8 = 0.54. It is not 0.54 divided by 0.475.

For a 16-by-16 image, split the rectangle from (2,2) to (10,10) into two triangles. With samples at half-integers, there are 8*8=64 covered samples. Test count==1 for each interior sample and count==0 elsewhere. Reversing the winding of both triangles must preserve that ownership result after orientation normalization.

Original lab assignment

Render two triangles sharing an edge and a receding checkerboard. Pass: no cracks under the documented fill rule, correct overlap ordering, and a visible comparison between affine and perspective-correct texture coordinates.

Assigned study

R19: rasterization, barycentric coordinates and texture mapping; R23: Sampling and Reconstruction.

Focused study: R19: coverage and perspective interpolation. R23: Sampling and Reconstruction. Keep a derivation and the numeric fixture with the code.

Check your understanding

1. What is the sum of barycentric coordinates?
2. When does perspective-correct interpolation reduce to affine interpolation?
3. Why specify a shared-edge coverage rule?
4. Compute perspective-correct u for weights (0.2,0.3,0.5), w=(1,2,4), u=(0,1,1).
5. Why is affine interpolation of z_ndc not the same mistake as affine interpolation of UVs?

LESSON 21.4 / MODULE 21

Rays, intersections and acceleration structures

> **Learning outcome:** Build a second visibility method and compare its costs with rasterization.

Rays ask the visibility question in the opposite direction

Rasterization projects surfaces onto a target. Ray casting starts at a sample and asks what surface is first encountered. Write a ray as r(t)=o+t*d over an explicit interval [tmin,tmax]. For a camera ray, d typically points into the scene, but its magnitude is not required to be one. Distances and ray parameters coincide only when the direction is unit length in the chosen coordinate units. Scaling d changes the numerical t of the same hit.

For a sphere centered at c with radius r, substitute the ray into |o+t*d*-c|^2=r^2. *With m=o-c, the quadratic coefficients are a=d dot d, b=2*(m dot d), and c0=m dot m-r^2. Reject zero-length directions. A negative discriminant misses; otherwise select the nearest root inside the stated interval. Tests must include rays starting inside the sphere, tangent rays and intersections behind the origin. For production-scale numeric ranges, use a stable quadratic formulation rather than assuming the elementary formula never suffers cancellation. [R23]

Bounding boxes reject groups conservatively

An axis-aligned box is the intersection of three slabs. On each nonparallel axis, compute the two t values at its bounds, sort them and intersect the resulting interval with the current ray interval. A zero direction component requires a special case: the origin must already lie within that axis's slab, otherwise it misses. Do not multiply zero by an infinite reciprocal and hope that NaN comparisons will behave as intended.

A bounding-volume hierarchy, or BVH, stores conservative bounds around primitive groups. A ray missing a node's bounds can skip its entire subtree. A first implementation can split sphere centers at the median along their widest axis. Leaves store small primitive lists. Correctness comes from conservative enclosure and exhaustive traversal of intersected candidate nodes, not from the sophistication of the splitting heuristic. [S04]

Keep an independent oracle

Retain a brute-force query that visits every primitive and returns the nearest valid hit. Compare hit/miss, primitive identity and t to BVH results over deterministic random rays. Equal-distance ties need a stable rule, such as lowest primitive ID within an explicitly stated tie policy. Record both primitive-test counts and wall-clock query time; a lower count is not itself proof of lower time in a high-overhead implementation.

Build time belongs in the report. A static scene queried by millions of rays may justify a slower builder, while frequently changing objects may favor a simpler build, a refit or a different structure. Overlapping bounds can force many node visits; do not promise logarithmic worst-case traversal.

Relationship to the engine

Use ray casting for a small reference image or object picking, not as an excuse to postpone the rasterizer. The two methods expose different mistakes. An incorrect camera convention can affect both, so retain direct numerical fixtures as well as image agreement. The package includes a small sphere BVH reference in Python and ray/box tests; robust triangle intersection and a high-performance native BVH remain valuable implementation extensions.

Worked example

The original ray has o=(0,0,0), d=(0,0,2). At t=3, the point is (0,0,6). The traveled distance is 6 coordinate units, not 3.

For a sphere at (0,0,5) with radius 1 and ray o=(0,0,0), d=(0,0,1), the equation is t^2-10t+24=0. Its roots are 4 and 6, so the nearest positive hit is t=4. With d=(0,0,2), the roots become 2 and 3 while the first hit position stays (0,0,4).

For box min=(-1,-1,4), max=(1,1,6), the unit +Z ray from the origin has parallel x/y directions, both origins inside their slabs, and a z interval [4,6]. The same ray from x=2 misses immediately because its x origin is outside a parallel slab. These fixtures should pass before adding hierarchy traversal.

Original lab assignment

Implement sphere or triangle intersections and a small BVH. Pass: accelerated and brute-force results agree within a defined tolerance over randomized rays, including misses and grazing cases; report build and query costs separately.

Assigned study

R23: Shapes and Primitives and Intersection Acceleration.

Focused study: S04: BVH construction and traversal. R23: sphere intersections and numerical robustness. Distinguish build and traversal evidence.

Check your understanding

1. When is ray parameter t directly a distance?
2. What point occurs in the worked example at t=3?
3. Why retain the brute-force implementation?
4. A sphere hit occurs at t=4 with unit direction. What t reaches the same point when the direction is doubled?
5. Why can a BVH with heavily overlapping node boxes approach brute-force behavior?

Module 21 review

1. Why should the renderer keep an image byte test even when screenshots look right?
2. Can a triangle with every original vertex outside the frustum still have visible area?
3. Which attributes use reciprocal-w interpolation, and what is the depth exception here?
4. What does a BVH correctness test compare besides execution time?
5. Why not promise exact GPU image equality for the CPU reference?

Project handoff

Retain the cpu reference renderer evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Module 22 — Explicit GPU programming and graphics APIs

Prerequisites: Module 21 plus Modules 18 and 20. Choose one compatible backend, not all at once.

Planning allocation: 90-140 planning hours, including the project's work and review.

Lesson	Outcome
22.1 — GPU work submission, resources and pipeline state	Separate CPU recording from asynchronous GPU execution.
22.2 — Buffers, shaders, layouts and resource binding	Match data representations across CPU and GPU boundaries.
22.3 — Synchronization, hazards and frames in flight	Express dependencies without serializing the entire renderer.
22.4 — Presentation, resizing and device capability policy	Treat the window system as a changing environment rather than a fixed image.

Original module project

Implement a GPU backend for the reference scene. Query device capabilities, enable available validation, document the toolchain and handle resize and shutdown safely. A CPU backend remains a valid fallback while hardware access is arranged.

Integrating milestones

1. P22-A: capability report, clear target and validation capture.
2. P22-B: geometry, transform layout and bounded compute readback.
3. P22-C: compute-to-draw dependency and two reusable contexts.
4. P22-D: resizing, loss policy and failure-safe teardown.

Evidence to retain

- Actual backend/toolchain record; byte layout table; shader sources; readback results.
- Dependency ledger; frame-context stress log; resize/minimize matrix; pending hardware markers where applicable.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 22.1 / MODULE 22

GPU work submission, resources and pipeline state

> **Learning outcome:** Separate CPU recording from asynchronous GPU execution.

Two processors, several timelines

A graphics program does not call a function and receive a finished image. The CPU constructs resources and records commands, then submits work that the GPU executes asynchronously. Separate the application timeline, command-recording lifetime, queue submission and completion. The return from a submit call is not evidence that a buffer is no longer in use.

Describe an initial frame as acquire target, record clear/draw operations, submit, present and eventually observe completion. Some APIs expose image acquisition directly; browser WebGPU manages more of the presentation machinery. The conceptual distinction still matters. A frame can be recorded but unsubmitted, submitted but incomplete, or complete but not yet visibly presented. An event loop must also remain responsive while these states change. [S05, S06]

Objects have different jobs

The device represents a selected implementation and enabled capability set. Buffers and textures provide storage. Views describe how storage is interpreted. A pipeline packages compatible shader stages and fixed-function state. Bindings connect shader-visible slots to particular resources and ranges. Command encoders or command buffers record operations, while queues execute submissions. A window or canvas surface is an output interface, not the device itself.

Ownership follows the asynchronous consumer. A CPU object going out of scope may destroy its wrapper while GPU commands still reference the underlying allocation. In native APIs, arrange deferred destruction after the relevant completion point. In managed APIs, follow their explicit object validity and destruction rules; do not infer the exact native lifetime contract from JavaScript garbage collection. The package's ownership ledger models completion serials on the CPU; it cannot validate a driver. [S06, S09]

Choose one backend deliberately

The canonical routes are Vulkan, Direct3D 12 or Metal on a compatible platform. The included WebGPU example is an additional teaching bridge for command recording, shaders, resource binding and bounded compute; it is not a substitute for learning native barriers when native engine work is the target. Select one route for the module project. Do not spend the volume implementing every API abstraction before you can show a validated clear operation.

Record OS, adapter, driver where exposed, SDK/browser build, shader compiler and enabled features. Query rather than assume limits and formats. An integrated GPU can be sufficient for these small labs; no performance claim follows merely from a marketing name. A software-only route remains valid while access is unavailable, but its report must say GPU validation pending.

Start with observable failure

Create a labeled device and target, enable available validation, and capture errors in a file or a visible log. Then clear to an unmistakable color. Check creation failures, a missing adapter and shutdown before adding

geometry. The useful first artifact is a capability report plus a correctly attributed clear operation, not a complex image with unexplained warnings.

The WebGPU example intentionally uses no optional advanced feature. It reports its actual device limits and failure messages when run. It is supplied as source; the package's automated CPU checks do not turn that source into hardware-tested evidence. This distinction is part of professional engineering, not a reason to skip the experiment.

Worked example

Suppose the CPU finishes recording at 1.2 ms, submit returns at 1.3 ms, the GPU starts at 1.5 ms and finishes at 5.8 ms. The CPU recording cost is 1.2 ms under that timer scope. Submission latency is roughly 0.1 ms. GPU execution is 4.3 ms only if those GPU timestamps were actually measured on a compatible clock. A CPU timer around submit cannot establish the 4.3 ms number.

A vertex buffer referenced by that submission may not be overwritten or freed merely because submit returned at 1.3 ms. Its last-use token corresponds to completion at 5.8 ms in this hypothetical timeline. A screenshot appearing at 16.7 ms would include presentation scheduling as well, and does not make the GPU work take 16.7 ms.

Write a lifetime ledger with columns resource, owner, last submission, completion test and retirement action. Populate it for the target, vertex buffer, uniform buffer and command storage before the first animated frame.

Original lab assignment

Create a window or offscreen target and report supported capabilities. Pass: render a clear color, capture validation output and explain which objects belong to CPU recording versus GPU execution.

Assigned study

R21: Introduction and Drawing a Triangle; alternatively R24 or R25 for one native platform; R22: SDL3 platform setup.

Focused study: S05: frames in flight. S06: device, encoding and queue mental model. R21/R24/R25: one selected native backend setup.

Check your understanding

1. Does queue submission mean the GPU has finished?
2. Why can immediate buffer destruction be unsafe?
3. Must a beginner implement three graphics APIs simultaneously?
4. A submit call returns after 0.2 ms. Can you conclude the GPU rendered in 0.2 ms? Explain.
5. A requested optional feature is absent. Give two valid policy responses and one invalid response.

LESSON 22.2 / MODULE 22

Buffers, shaders, layouts and resource binding

> **Learning outcome:** Match data representations across CPU and GPU boundaries.

Host and shader share bytes, not field names

A shader reads a buffer according to its own layout rules. A C++ struct with identically named members can have different offsets, padding, array strides or matrix interpretation. Write the byte contract before uploading. Distinguish vertex attribute layout, storage-buffer layout and uniform-buffer layout: they are separate contracts even inside the same API.

For the browser example, vertex data is two vec4 values: position at byte 0 and linear color at byte 16, giving a 32-byte vertex stride. The transform uniform is a column-major mat4x4, occupying 64 bytes. In C++, static_assert and offsetof can check the matching host representation; in JavaScript, explicitly index the Float32Array. A correct size with a transposed matrix still produces wrong geometry. [S01]

A particularly useful WGSL example is a struct containing position:vec3 followed by scale:f32. The vector has alignment 16 and size 12, so scale may occupy offset 12 and the struct can occupy 16 bytes. But an array of vec3 has a 16-byte element stride. Do not summarize this as either all vec3 values are 12 bytes or all are 16 bytes. Consult the rule for the actual address space and aggregate. Dynamic binding offsets have an additional device-limit alignment requirement. [S01]

Bindings are a resource interface

A resource binding associates a shader location with a buffer or texture and a permitted range or view. Pipeline layouts constrain type, visibility and compatibility. Binding the right data to the wrong group or slot is not repaired by renaming a variable. Treat binding identifiers as part of the interface version and keep a table beside the shader.

The upload path must also state whether the API copies source bytes immediately, stages them, or permits persistent mapping under specified synchronization. Do not mutate CPU staging storage on the assumption that every API behaves like the one used in a previous lesson. A mapped GPU buffer generally needs a deliberate ownership transition before execution can consume it. [S06]

Compute covers work in groups

For N elements and group size G, launch ceil(N/G) groups. Global invocation i operates only when i<N. For N=1000 and G=64, sixteen groups create 1,024 invocations, so the last 24 must do no element access. Reject invalid sizes and query the selected device's limits rather than assuming any group shape is portable.

Use a simple bounded function, such as output[i]=2*input[i]+1, and read the result back. A visible triangle alone does not check compute correctness. For float comparisons, choose an expected tolerance based on the operations; for exactly representable small integers in f32, exact comparison can be appropriate. Check N=0, 1, G-1, G and G+1. Zero work should be handled explicitly rather than creating a zero-size resource and hoping the API accepts it.

Build a diagnostic ladder

Test raw bytes, then a known transform, then resource bindings, then readback. Only after these work should compute write data consumed by a draw. The included browser demonstration computes vertex output and

draws from that storage, with a separate readback check. Hardware execution remains a learner validation step.

Worked example

For a position/scale storage record, the chosen WGSL layout is position at 0, scale at 12, total 16. Two records occupy 32 bytes. A different record with position:vec3 followed by color:vec3 places color at offset 16 and has total size 32 because the aggregate is rounded to its 16-byte alignment.

For 1,000 elements and 64 threads per group, (1000+63)//64=16. The shader guard is if (i >= count) { return; }. A test with 1,025 elements would need 17 groups, making off-by-one assumptions visible.

For a column-major translation matrix, place tx,ty,tz in the first three elements of the last column. Multiplying the homogeneous point (0,0,0,1) must produce (tx,ty,tz,1). Test this special point before testing a rotating object, where a layout error is harder to diagnose.

Original lab assignment

Upload a transform and vertex colors, then add a bounded compute operation. Pass: host/shader layouts are documented, a readback test checks results, and non-multiple work sizes remain correct.

Assigned study

R21: vertex buffers, uniform buffers and compute; R37: execution configuration and memory organization, only for the optional CUDA route.

Focused study: S01: alignment, size, structure and array layout; compute built-ins. S06: buffer mapping and bindings. R21: upload and compute route.

Check your understanding

1. Why can matching field names still produce a layout bug?
2. How many 64-thread groups cover 1,000 elements?
3. What does a resource binding establish?
4. A WGSL storage struct contains vec3 position and f32 scale. What are the member offsets and total size?
5. How many 64-thread groups cover 1,025 elements, and why is a bounds guard still required?

LESSON 22.3 / MODULE 22

Synchronization, hazards and frames in flight

> **Learning outcome:** Express dependencies without serializing the entire renderer.

Name the dependency before choosing a primitive

A producer writes data that a consumer later reads: this is a read-after-write hazard. Execution order is necessary, but some APIs also require an explicit memory dependency making those writes available and visible in the consumer's access domain. A write-after-read dependency needs the old read to finish before new writes; a write-after-write dependency must order the writes appropriately. Start with resource, range, producer stage/access and consumer stage/access. Only then select the API mechanism. [S07]

In a Vulkan single-queue example, compute storage writes consumed as vertex input require a dependency from compute-shader writes to vertex-input reads of the correct range. A color attachment written by a render pass and sampled in a later fragment shader needs different stages and access masks, and possibly an image layout transition. A shader-stage barrier cannot be guessed from the word texture. Cross-queue work also requires appropriate submission synchronization and, where applicable, queue-family ownership transfer.

Direct3D 12 exposes resource states and barriers with its own rules. Metal has resource tracking and synchronization modes that must be read in their native context. WebGPU validates usage scopes and manages many transitions automatically; sequencing separate legal compute and render passes is not evidence that a native barrier can be omitted. The abstract dependency is shared; the exact API expression is not. [S09, S10, S06]

Frames in flight need reusable ownership units

A frame context groups mutable resources that will be reused together: command storage, per-frame uniform storage and relevant temporary allocations. Give each context a completion token. Before resetting or overwriting context k, establish that the submission last using k has completed. Two contexts allow useful overlap without treating the entire renderer as globally idle after every operation. Static read-only resources need not all be duplicated.

Frame index and acquired presentation-image index are different. In Vulkan, presentation's wait on a render-finished binary semaphore can outlive the graphics submission fence. A fence for a frame is therefore not, by itself, proof that its present-wait semaphore is reusable. Follow a supported lifetime scheme, such as render-finished semaphores indexed by acquired swapchain image, and consult the relevant presentation-completion rules. This is a common bug in otherwise plausible two-frame tutorials. [S05, S08]

Completion controls retirement

Suppose a texture's last use is submission 12. Replacing its handle at submission 13 does not immediately free the old allocation. Queue it for retirement and release it when the completion ledger includes 12, with every queue that used it accounted for. The monotonic serial model in this package is deliberately single-queue. A multi-queue renderer needs a completion set or equivalent dependency proof, not a single global number invented by the CPU.

Stress the boundary

Slow the GPU path or increase work so premature reuse becomes visible. Alternate contexts, resize, reload a resource and exit while work is outstanding. Validation is necessary but not a complete proof; some logical

lifetime bugs produce legal commands using wrong data. Pair validation logs with frame IDs encoded into output and readback where practical. Use coarse waiting for initial correctness and controlled teardown, then replace unnecessary waits with precise dependencies.

Worked example

Context A was last submitted as serial 10, context B as 11, and the completed serial is 9. Neither can be reset. Once completion advances to 10, A may be reused for serial 12 while B remains busy. Merely calculating frame_number % 2 does not prove the chosen context is free.

For a compute-to-vertex-buffer path, the dependency ledger reads: resource generated_vertices; producer compute shader / storage write; consumer vertex input / attribute read; same queue; buffer range [0,vertex_bytes). A shader-storage read in a vertex shader would be a different consumer access than fixed-function vertex input. Name the actual consumption mechanism.

If three swapchain images coexist with two CPU frame contexts, valid image indices are 0,1,2 while context indices are 0,1. An image-indexed array of length two is an immediate bounds bug, and a frame-indexed present semaphore can have a subtler reuse bug even when every index stays in range.

Original lab assignment

Implement a compute-to-draw dependency and two reusable frame contexts. Pass: validation reports no relevant hazards and a stress test never resets a context still used by the GPU.

Assigned study

R21: synchronization, frames in flight and resource transitions; use R24 or R25 equivalents for the selected backend.

Focused study: S07: compute/graphics dependencies. S05: frame contexts. S08: swapchain semaphore reuse. S09/S10: native backend equivalents.

Check your understanding

1. Are execution ordering and memory visibility identical?
2. When may a frame context be reused?
3. Why is waiting for the whole GPU after each command a poor final design?
4. Context A last used serial 10 and completed serial is 9. Is frame_number modulo two sufficient to reuse A?
5. Why can a graphics submission fence be insufficient to prove that a Vulkan presentation-wait semaphore is reusable?

LESSON 22.4 / MODULE 22

Presentation, resizing and device capability policy

> **Learning outcome:** Treat the window system as a changing environment rather than a fixed image.

The drawable is changing state

A window can be resized, minimized, moved between displays or closed during initialization. Its logical size may differ from its framebuffer pixel size. Query the drawable size in pixels for render-target allocation and aspect ratio; do not assume a high-density display uses one pixel per window-coordinate unit. A zero width or height is a valid state to handle, not a valid render-target extent for the normal path. [S11]

Write a surface state machine: unavailable, ready, resize pending, rebuilding, lost and stopping. Rendering only consumes a ready surface. A resize event records the new desired extent; rebuild at a controlled boundary after handling resources still in flight. Do not rebuild independently inside an event callback that can race the render path.

Recreate a dependency set, not just a viewport

Size-dependent resources may include depth, multisample color, post-processing targets, history buffers, framebuffers and projection data. Temporal history often becomes invalid when size or projection changes. Build the new set transactionally: validate extents, allocate compatible resources, then publish it. Retire old resources only after their last use. Under allocation failure, preserve a known valid state or stop with an explanation.

In Vulkan, handle out-of-date and suboptimal results according to policy, including acquisition and presentation paths. A common deadlock arises when a fence is reset and acquisition then returns early without a submission that will signal it. Delay reset until the code is committed to submission, or otherwise repair the state. A minimal renderer may wait for idle during recreation; make that deliberate, not a per-frame habit. [S12]

Capabilities become product policy

Divide features into required, optional and disabled. Required features gate startup. Optional features select a tested fallback when absent. Disabled features are excluded even if the adapter advertises them, because this build has no validated implementation. Record enabled device capabilities rather than merely the maximum capabilities advertised by the adapter.

Shader compilation and pipeline creation can fail. Device loss is a distinct condition from window resize: a lost device may invalidate the entire device-owned resource set. The included browser bridge stops, reports loss and requires reinitialization through a fresh page run; it does not claim seamless recovery. An engine may choose a controlled restart rather than implementing transparent recreation of every subsystem.

Teardown is an ordinary execution path

Stop scheduling new work, cancel pending loads, wait or otherwise satisfy the required completion conditions, release resources in an ownership-safe order, and close platform objects. Test shutdown after each initialization stage, not only after a successful frame. Use an initialization record or RAII owners so cleanup acts on exactly the objects that exist.

Your compatibility matrix is evidence, not a wish list. Record actual runs separately from plausible support. A software fallback with honest limitations is more useful than a GPU path whose warnings and failures have been edited out of the report.

Worked example

A window reports logical size 800-by-600 but drawable size 1600-by-1200. A depth target allocated at logical resolution would not match a full-resolution color target. At four bytes per depth sample, the correct single-sample depth allocation is 1600*1200*4=7,680,000 payload bytes, before API alignment and metadata.

A stress sequence is (1280,720), (0,0), (800,600), (1600,1200), close. The zero extent should suspend size-dependent rendering without dividing by height or allocating an ordinary zero-size target. Restoration invalidates size-dependent history and creates the new target set.

In the early-return fence bug, wait succeeds, reset makes the fence unsignaled, acquisition returns out-of-date, and the function returns. The next wait can hang forever because no submission was issued to signal that fence. Draw the state transition to explain why the reset belongs after a successful acquisition decision.

Original lab assignment

Stress resize, minimize, restore and orderly shutdown. Pass: no stale size-dependent resources, no division by zero, and a readable capability report explains enabled features and fallbacks.

Assigned study

R21: swapchain recreation and device selection; R22: window events; R24 or R25: presentation and resource lifetime.

Focused study: S11: pixel drawable size. S12: swapchain recreation and fence-reset placement. S06: device loss and canvas lifecycle.

Check your understanding

1. Why can resizing require more than changing the viewport?
2. What should happen for a zero-height drawable?
3. What is the purpose of an explicit feature policy?
4. An 800-by-600 logical window has scale factor two in both dimensions. What drawable extent and four-byte depth payload result?
5. Explain how resetting a fence before an acquire-out-of-date early return can deadlock the next frame.

Module 22 review

1. Why can correct submission order still be an incomplete native synchronization argument?
2. Are WebGPU implicit transitions proof that Vulkan barriers are unnecessary?
3. What separates frame-context indices from presentation-image indices?
4. Why should GPU results include readback instead of only a triangle?
5. What can a CPU-only run honestly establish for this module?

Project handoff

Retain the one validated gpu route evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Module 23 — Lighting, materials and real-time image quality

Prerequisites: Modules 07, 09-10, 21-22.

Planning allocation: 90-140 planning hours, including the project's work and review.

Lesson	Outcome
23.1 — Radiometry, reflectance and physically based materials	Explain how lighting models approximate light transport.
23.2 — Textures, mipmaps, filtering and tangent-space detail	Control aliasing while preserving the meaning of texture data.
23.3 — Shadows, light selection and rendering-path tradeoffs	Choose a lighting architecture from scene and bandwidth requirements.
23.4 — HDR, tone mapping, temporal reconstruction and validation	Separate display mapping from reconstruction and simulation correctness.

Original module project

Create a material-and-lighting demonstration with a documented color pipeline. Compare image quality and frame cost at identical resolution and scene content; include reference images, not only attractive screenshots.

Integrating milestones

1. P23-A: linear-light diffuse and documented microfacet material.
2. P23-B: semantic mip filtering and tangent-handedness tests.
3. P23-C: one shadow map and an equal-quality lighting-path comparison.
4. P23-D: exposure, tone mapping and temporal failure/rejection sequences.

Evidence to retain

- Units/space table; raw linear fixtures; material grid; normal seam test.
- Buffer budget; light-count data; shadow-bias comparison; temporal sequence and error domain.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 23.1 / MODULE 23

Radiometry, reflectance and physically based materials

> **Learning outcome:** Explain how lighting models approximate light transport.

Name the quantity before choosing the formula

Radiant flux is power, measured in watts. Irradiance is incident power per area, measured in W/m^2. Radiance adds a directional density, measured in W/(m^2 sr). These distinctions keep a light's intensity, a surface's reflectance and the displayed pixel value from becoming interchangeable sliders. In this volume, RGB lighting is a fixed-primary approximation to spectral transport, not a claim to represent every spectrum exactly. [S13]

A bidirectional reflectance distribution function, or BRDF, relates incident radiance from one direction to reflected radiance toward another direction at a surface. Its units are inverse steradians. The rendering equation integrates incoming radiance times the BRDF and the incident cosine over the hemisphere, with emission added separately. For an ideal Lambertian reflector of reflectance rho, f=rho/pi. The pi normalizes the cosine-weighted hemisphere integral; dropping it changes energy, not just artistic brightness. [R23]

Build one explicit material model

Start with diffuse reflectance, then add a single documented microfacet model. This package uses the common GGX normal distribution, Schlick Fresnel and separable Smith masking-shadowing as a teaching model. Let r be perceptual roughness and alpha=max(rr,*0.001). Let h be the normalized halfway vector of the unit light and view directions. With nh=max(dot(n,h),0), D=alpha^2/[pi*(nh^2(alpha^2-1)+1)^2]. The specular BRDF is DGF/(4*nl*nv) for positive nl and nv. At or below the surface, return zero instead of dividing by a tiny or negative denominator. [S14]

Schlick's approximation uses F=F0+(1-F0)*(1-vh)^5. For an illustrative dielectric with index of refraction 1.5 against air, normal-incidence reflectance is ((1.5-1)/(1.5+1))^2=0.04. Do not make 0.04 a universal material constant. Metals generally require colored specular response and do not use the same diffuse lobe as the simple dielectric route. A metallic workflow blends parameters under a stated convention; its sliders are not arbitrary color operations. [S03]

Normals and frames are part of the material

Transform normals using the inverse transpose of the linear model transform when needed, then normalize. Nonuniform scaling is the counterexample that exposes using the position matrix directly. Tangent frames, UV handedness and light directions must all agree about space. Decide whether a quantity is world-space, view-space or tangent-space before taking a dot product.

Plausibility is not validation

Test zero light, zero reflectance, normal incidence, grazing angles and roughness limits. Guard nonfinite inputs and an undefined halfway vector when view and light oppose one another. Clamp only at documented model boundaries; clamping the final HDR result to one hides missing exposure handling and can conceal energy mistakes.

A microfacet implementation can be physically motivated without being physically exact. The simple single-scattering model can lose energy for rough surfaces; it is not a complete multiple-scattering solution. A

white-furnace experiment must use a correctly integrated environment and isolate the model's assumptions. Do not claim energy conservation because one shiny sphere looks convincing. Preserve raw linear values alongside the display image.

Worked example

For rho=0.5 and irradiance E=10 W/m^2, an ideal diffuse reflector produces outgoing radiance L=rho*E/pi, approximately 1.59155 W/(m^2 sr), independently of view direction for this idealized model.

For the teaching specular model at normal incidence with roughness r=0.5, alpha=0.25. Therefore D=1/(pi*0.25^2), approximately 5.09296. With G=1 and F0=0.04, the specular BRDF is 5.092960.04/4, approximately 0.05093 sr^-1. D exceeding one is not by itself an energy violation; it is a density whose relevant integral matters.

As a geometry test, a normal (1,1,0) under nonuniform scale diag(2,1,1) should transform proportionally to (0.5,1,0), then be normalized. Transforming it as a position direction gives (2,1,0), a different orientation. Verify the transformed normal remains perpendicular to transformed tangent directions.

Original lab assignment

Implement diffuse lighting and one documented microfacet model. Pass: test a constant environment, grazing angles and roughness limits, and state approximations rather than labeling any shiny shader physically exact.

Assigned study

R23: Radiometry, Reflection Models and Light Transport; R19: shading.

Focused study: S13: radiometric quantities. S14: microfacet distributions. S03: metallic-roughness parameter semantics. R23: diffuse and light transport.

Check your understanding

1. What does a BRDF relate?
2. What is the Lambertian BRDF for reflectance rho?
3. Why document roughness encoding?
4. For rho=0.5 and E=10, compute ideal diffuse outgoing radiance.
5. Why does a BRDF or normal-distribution value above one not automatically imply energy creation?

LESSON 23.2 / MODULE 23

Textures, mipmaps, filtering and tangent-space detail

> **Learning outcome:** Control aliasing while preserving the meaning of texture data.

Textures are sampled signals

A texture is a discrete signal evaluated over a surface footprint. When many texels project onto one pixel, selecting a single nearest sample does not summarize that footprint; it aliases. Bilinear interpolation smooths between nearby samples but does not by itself provide the required large-footprint prefilter. A mip pyramid stores progressively filtered resolutions so minifying samples can use a coarser representation. [S15]

A full square power-of-two pyramid has N^2+(N/2)^2+...+1 texels. The geometric series approaches 4/3 of the base size. This factor is about texel count under those assumptions; row alignment, block compression, array layers and implementation allocation rules can change actual memory. Count the real level dimensions when sizes are not powers of two.

Filter according to meaning

Decode color into the chosen linear-light working representation before averaging it for mip generation. Then encode or store the result according to the runtime texture format. Averaging nonlinear color bytes changes the represented energy. Conversely, roughness, occlusion and normals are not color and should not receive an automatic sRGB decode. Normal-vector filtering may require renormalization and a policy for the lost subpixel distribution; renormalization alone is not a full antialiasing model for specular response. [S03, S15]

Alpha needs its own contract. Straight-alpha and premultiplied-alpha content require different filtering and compositing treatment. For cutout foliage, ordinary averaging can change apparent coverage at distant mip levels. Document whether the experiment preserves alpha coverage; do not silently call any averaged mask equivalent.

A tangent frame has handedness

A tangent-space normal is decoded, transformed by the tangent/bitangent/normal frame and normalized. If tangent t and geometric normal n are orthogonalized, a common bitangent construction is b=sign*cross(n,t), where sign records handedness. Mirrored UV islands can flip that sign. A tangent basis created by one convention and a normal map baked using another can produce a seam despite individually valid vectors. [S03]

Degenerate UV parameterizations cannot provide a meaningful inverse mapping for a tangent basis. Detect them and choose a documented fallback rather than dividing by a near-zero UV determinant. Include a mirrored-UV seam and nonuniformly scaled object in the validation scene.

Isotropic and anisotropic footprints

A receding floor produces a long, thin texture footprint. An isotropic mip choice must compromise between aliasing along one direction and unnecessary blur along the other. Anisotropic filtering samples a more elongated footprint within hardware and sampler limits. It is a quality/cost decision, not a guarantee that every high-frequency pattern can be perfectly reconstructed.

Begin with nearest and bilinear sampling, then mip selection, then compare supported anisotropy. Keep camera, resolution, texture and motion identical. Record the selected level or footprint estimate in a debug view. A nicer still screenshot can hide shimmer during motion, so include a fixed camera path as well as a single image.

Worked example

A 256-by-256 base texture has 65,536 texels. The full chain is 65,536+16,384+4,096+1,024+256+64+16+4+1=87,381 texels. At four bytes per texel, that is 349,524 payload bytes, before alignment or metadata. The exact factor is 87,381/65,536, slightly less than 4/3.

For color samples black and white, linear averaging gives 0.5, which becomes approximately 0.7354 under sRGB encoding. For a normal channel storing 0.5, the intended decoded signed component may be zero; applying sRGB decoding first would yield about 0.214 and then approximately -0.572 after remapping, an entirely different vector component.

For n=(0,0,1), t=(1,0,0), cross(n,t)=(0,1,0). A mirrored island with sign=-1 needs b=(0,-1,0). Omitting that sign flips how the normal map's second axis is interpreted.

Original lab assignment

Render a receding textured floor with and without mipmaps and inspect a normal-mapped seam. Pass: report texture storage including mip levels and explain why non-color data needs its own filtering policy.

Assigned study

R23: Textures and Sampling and Reconstruction; R21: texture images, samplers and mipmaps.

Focused study: S15: image-texture filtering and mipmaps. S03: color/data and tangent conventions. R23: sampled reconstruction.

Check your understanding

1. What problem do mipmaps primarily address?
2. What is the approximate total mip-chain storage factor?
3. Why can mirrored UVs break normal mapping?
4. How many texels and RGBA8 payload bytes are in the full 256-by-256 mip chain?
5. For n=(0,0,1), t=(1,0,0) and handedness -1, what bitangent is required?

LESSON 23.3 / MODULE 23

Shadows, light selection and rendering-path tradeoffs

> **Learning outcome:** Choose a lighting architecture from scene and bandwidth requirements.

A shadow map is another visibility computation

Render depth from the light's viewpoint. For a visible camera sample, transform its position into the same light clip convention and compare its depth with the stored light-visible depth. Samples outside the light's coverage need an explicit policy, not an accidental clamped texture lookup. A shadow map is finite-resolution evidence of visibility from one light, not a complete solution to global illumination.

Self-shadowing can arise from finite precision and different sampling of the receiver. A small depth bias can reduce this acne, but excessive bias detaches the shadow from the object, often called peter-panning. The sign and scale depend on the chosen depth convention, comparison and projection. Slope-sensitive bias and normal offsets address different geometric effects. First test a flat receiver and a small hovering object so the tradeoff is visible. [R19]

Percentage-closer filtering averages nearby depth-comparison results. It does not simply blur the depth texture and then compare once; those operations are not equivalent. Cascades, temporal stabilization and advanced soft-shadow methods are extensions after the basic comparison passes. A directional-light shadow should not swim merely because a camera moved a tiny amount without an explained projection change.

Choose where lighting work happens

In a simple forward renderer, geometry is drawn and shaded with selected lights directly into the color target. Its simplicity is useful for the small engine. In deferred shading, a geometry pass stores surface attributes in a G-buffer and a later pass performs lighting. That can reduce repeated shading for certain scenes, but introduces storage and bandwidth and complicates some combinations of transparency, material diversity and multisampling.

Tiled or clustered light selection assigns candidate lights to screen-space or screen/depth regions. It can benefit either forward or deferred rendering by reducing irrelevant light evaluations. Its cost includes building the lists, storing them, managing overflow and looking them up. Scene-dependent rejection is the mechanism; the word clustered is not itself a performance result. [R19]

Build a cost model you can test

Estimate pixels, light candidates, render-target bytes and passes. Then measure the declared scene. For each path, hold resolution, camera, visible content, light set and quality constant. A path that quietly drops lights is not a fair speed comparison. Overflow handling is part of correctness: a fixed-size list must either use a supported fallback, spill safely or report a documented quality reduction.

Track transparent geometry separately. A deferred opaque pass does not automatically solve ordered translucent composition. Keep the initial scope small: opaque objects, one directional shadow and a controlled point-light set. Add a transparent test only with a declared path and limitations.

Interpret the evidence carefully

Separate CPU list-building time from GPU lighting time and full-frame time. Report distributions, not only one number. A simplified arithmetic bandwidth estimate is a planning bound, not measured memory traffic; caches, compression, tiling and implementation details matter. The module's outcome is the ability to justify a renderer configuration and demonstrate its limits, not to prove that one architecture wins every scene.

Worked example

Suppose a 1920-by-1080 G-buffer stores 16 bytes per sample across its chosen attachments. One complete logical write is 1920*1080*16=33,177,600 bytes, about 31.64 MiB. At 60 frames per second, that alone is roughly 1.99 GB/s of logical payload. Reading it once for lighting doubles this simplified traffic estimate; depth, final color, cache behavior and compression are not included.

With 256 lights and 2,073,600 visible pixels, naive all-lights shading evaluates about 530.8 million pixel/light candidates before rejection inside the shader. If clustering reduces the average candidate set to 12, the analogous count is 24.9 million, plus list-building and lookup overhead. That is a hypothesis for measurement, not a guaranteed 21-fold speedup.

For conventional increasing depth, compare receiver_depth-bias <= shadow_depth. Increasing a positive bias makes the test more permissive and can detach contact shadows. Reverse depth needs a correspondingly consistent rule rather than copying this sign blindly.

Original lab assignment

Implement shadow mapping and compare a simple forward path with tiled light selection or a small deferred prototype. Pass: disclose buffer sizes, light counts, transparency limitations and measured frame costs.

Assigned study

R19: real-time rendering and shadows; R23: Lights and Light Transport.

Focused study: R19: shadow mapping and real-time rendering paths. R23: lights and visibility. S07: depth-write to shader-read dependency for a native shadow map.

Check your understanding

1. Why can excessive shadow bias be visible?
2. Does deferred rendering remove bandwidth cost?
3. What determines whether clustered lighting helps?
4. What is the logical payload of a 1920-by-1080, 16-byte-per-pixel G-buffer write?
5. A tile light list has capacity 64 but receives 80 lights. What makes overflow handling an engineering requirement rather than an optional optimization?

LESSON 23.4 / MODULE 23

HDR, tone mapping, temporal reconstruction and validation

> **Learning outcome:** Separate display mapping from reconstruction and simulation correctness.

HDR storage and HDR display are different claims

An internal high-dynamic-range buffer can store scene values above one without implying that the application drives an HDR display. The first engine can render into floating-point linear RGB, apply exposure and a tone mapper, and present a standard-dynamic-range sRGB output. State this precisely. Clamping intermediate lighting to [0,1] destroys highlight information before the display mapping can use it.

Exposure scales the scene-linear value; an exposure change of e stops multiplies it by 2^e. A simple teaching tone mapper is T(x)=x/(1+x) for nonnegative x. It compresses highlights but is not a complete photographic or color-appearance model. Applying it independently per RGB channel can shift chromatic relationships. Document that limitation instead of naming the result physically correct. Output transfer encoding comes after tone mapping, exactly once. [R23]

Temporal reuse needs correspondence

Temporal accumulation combines a current sample with history, often Hnew=(1-w)*C*+*w*Hreprojected. The history must represent the same visible surface or signal. Reprojection uses current and previous transforms or motion vectors under an explicit pixel/UV convention. Camera motion and object motion are distinct sources; omitting either can put history on the wrong object.

A camera cut breaks the correspondence. Disocclusion reveals surfaces whose useful history may not exist. Resolution changes, projection changes, large exposure changes and changes to jitter policy can also invalidate or require transforming history. Compare depth, identity or other available signals conservatively, and reject or reduce history when correspondence is not credible. An old sample is not inherently better because it was expensive to compute. [R19]

Jitter and motion are not interchangeable

Subpixel camera jitter varies sample locations so multiple frames can gather additional information. Motion vectors describe movement of the represented surface between frames. Their conventions must account for whether jitter is included or excluded. Incorrectly mixing these transforms can cause a static scene to blur or crawl. Begin with an unjittered static accumulation experiment, then add a known subpixel pattern and a controlled camera path.

Neighborhood clipping or clamping can suppress ghosting but can also bias the result or remove detail. Treat it as a heuristic with visible tradeoffs. A basic temporal experiment is not equivalent to a production reconstruction system, frame generation or a vendor upscaler.

Validate space and time

A still image may look cleaner while motion produces long trails. Preserve both a fixed-frame image comparison and a repeatable sequence containing a camera cut, fast object motion, disocclusion and thin geometry. Compare to a more heavily sampled reference when possible. State whether error is measured in linear scene values, tone-mapped linear values or encoded display values; the choice changes the meaning of the number.

A useful test makes history obviously wrong: current value 0.2, old value 1.0 and a camera cut. With history weight 0.9, accidental reuse yields 0.92. Correct rejection yields 0.2. This numerical fixture cannot validate all visual quality, but it decisively detects a missing reset path.

Worked example

For scene-linear luminance x=4 and exposure -1 stop, the exposed value is 4*2^-1=2. The teaching tone mapper gives 2/(1+2)=2/3. sRGB encoding gives approximately 0.8360. Calling 2/3 the final encoded pixel would skip the output transfer step.

For C=0.2, H=1.0 and history weight w=0.9, the blend is 0.10.2+0.91=0.92. At a camera cut, reject history so the result is C=0.2. With a valid static history of H=0.21, the same weight gives 0.209, illustrating why correspondence determines whether averaging is useful.

For a 10-by-10 RGB comparison with absolute channel errors totaling 6.0, mean absolute error is 6/(10103)=0.02. Label the color domain and reference image; without them, the number is not reproducible or interpretable.

Original lab assignment

Add exposure, a documented tone mapper and a basic temporal accumulation experiment. Pass: capture a camera cut and disocclusion test, show failure modes, and compare both still-image error and motion behavior.

Assigned study

R23: Film and Imaging and Sampling; R19: anti-aliasing and image processing.

Focused study: R23: film, imaging and sampling. R19: antialiasing and reconstruction. S03: distinguish display color encoding from data handling.

Check your understanding

1. Are tone mapping and correct light transport the same task?
2. Why reject history at a camera cut?
3. Why are still screenshots insufficient for temporal methods?
4. Apply -1 exposure stop and T(x)=x/(1+x) to scene value 4. What value reaches the output transfer encoder?
5. Current sample is 0.2, history 1.0, weight 0.9. What are the blended value and correct camera-cut value?

Module 23 review

1. Why distinguish radiance from irradiance in a shader notebook?
2. Why can a perfect normal-map seam on one mesh be insufficient?
3. Why is a lower light-list count not a measured speedup?
4. Does an HDR intermediate prove HDR display output?
5. Why preserve camera-cut and disocclusion failures in the report?

Project handoff

Retain the material and lighting laboratory evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Module 24 — Engine architecture, simulation and resource ownership

Prerequisites: Modules 12, 17-18, 21-23.

Planning allocation: 80-120 planning hours, including the project's work and review.

Lesson	Outcome
24.1 — Fixed updates, interpolation and overload policy	Separate simulation time from rendering time and real elapsed time.
24.2 — Entities, components, handles and data-oriented storage	Design identity separately from memory location.
24.3 — Allocators, task graphs and frame-resource lifetime	Make temporary storage and parallel dependencies observable.
24.4 — Subsystem boundaries, save formats and reloadability	Choose interfaces that make change and failure manageable.

Original module project

Build a small engine with a fixed-step world, generational entity handles, explicit resource ownership and an inspectable task graph. Keep the game rules separate enough to run automated headless tests.

Integrating milestones

1. P24-A: fixed ticks, interpolation and explicit overload accounting.
2. P24-B: generational identity and deferred structural edits.
3. P24-C: checked arena plus serial/parallel task-graph validation.
4. P24-D: schema migration, transactional loads and safe reload/shutdown.

Evidence to retain

- Tick-indexed input log; time invariant tests; entity mutation tests.
- Task trace and critical path; arena guard tests; save fixtures; stale reload and initialization-failure logs.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 24.1 / MODULE 24

Fixed updates, interpolation and overload policy

> **Learning outcome:** Separate simulation time from rendering time and real elapsed time.

Simulation time is a design choice

A rendering callback arrives according to scheduling, device load and presentation constraints. Game rules should not silently inherit that variable interval. Maintain a fixed simulation step h and an accumulator of accepted elapsed time. Add elapsed time, run complete steps while enough time remains, and retain the fractional remainder for rendering. The simulation advances in whole ticks, while rendering can interpolate between saved states. [R27]

Keep previous and current simulation snapshots. After completing available steps, alpha=accumulator/h should lie in [0,1) under the selected policy. Rendering lerp(previous,current,alpha) intentionally displays a state between two completed ticks, generally about one tick behind the latest real-time trajectory. It avoids predicting an uncomputed future state but introduces that interpolation delay. Camera or user-interface elements may need their own latency policy.

Inputs belong to ticks

Sample or receive input events with enough timing information to assign them consistently to simulation ticks. Do not apply the same press repeatedly merely because multiple catch-up ticks occur. Separate continuous state, such as a held movement direction, from discrete edges, such as a one-time jump press. Record tick-indexed input when building replay tests.

Fixed h is not a universal determinism switch. Random-number order, floating-point behavior, unordered iteration, task races, input assignment and external I/O can all change results. State the reproducibility level actually supported: same executable and input stream, same platform family, or cross-platform bit identity. A successful same-process test does not prove the strongest claim.

Overload needs an explicit response

An application that always tries to simulate all accumulated time can fall behind: catch-up work causes a longer frame, which creates still more catch-up work. Choose a policy. A teaching interactive policy clamps an extreme incoming frame interval, performs at most K steps, then discards whole remaining ticks while retaining a sub-tick remainder. Record every discarded interval. This sacrifices synchronization with real elapsed time to protect responsiveness; it must not be used silently in a network-authoritative simulation or scientific model.

Alternatives include slowing presentation, reducing optional work, pausing with a warning or running a headless simulation without dropping ticks. The correct policy depends on product requirements. A cap of eight steps is not automatically better than four; it changes the maximum compute burst and tolerated latency.

Measure three clocks

Report real elapsed time, accepted simulation time and dropped time separately. The invariant for the simple drop policy is old_accumulator+elapsed = steps*h + new_accumulator + dropped, with all terms in the same units. Use integer time units in small tests so fractional-millisecond arithmetic does not obscure the policy. The C++ reference uses a checked integer-microsecond budget helper.

Test normal cadence, a 100-ms stall, repeated stalls, zero elapsed time and invalid negative input. Then verify a world update with constant velocity. Keep a render interpolation test separate from the simulation state test; comparing rendered positions at different callback times is not the same as comparing states after the same tick count.

Worked example

With h=10 ms and accumulated time 25 ms, execute two steps and retain 5 ms. The interpolation fraction is 5/10=0.5, matching the original quiz. If previous x=1.0 and current x=1.2, render x=1.1.

Under a maximum of four steps, a 105-ms accumulator executes 40 ms of simulation. The 65-ms remainder contains six whole 10-ms ticks and 5 ms. The teaching drop policy records 60 ms dropped and retains 5 ms, so alpha=0.5. The accounting is 105=40+60+5.

For velocity 3 units/s and h=0.01 s, each tick adds 0.03 units. One hundred ticks advance 3 units under this simple integration rule. Ten calls carrying 100 ms each can produce a different tick count when the overload cap discards time, so do not expect the same result without considering the policy.

Original lab assignment

Implement an accumulator loop and inject long frames. Pass: simulation advances consistently under normal load, interpolation is bounded, and the overload behavior is tested rather than silently spiraling.

Assigned study

R27: Fix Your Timestep!; R20: game loops and timing. Use the article as a starting model, not a guarantee of determinism.

Focused study: R27: accumulator and overload model. R20: engine timing. Treat the worked policy as an explicit design choice, not universal advice.

Check your understanding

1. What fraction remains after two 10 ms steps from 25 ms?
2. Does a fixed timestep alone guarantee determinism?
3. Why must catch-up work be bounded or otherwise controlled?
4. Under the teaching drop policy, h=10 ms, max steps=4 and accumulated time=105 ms. Give steps, remainder, dropped time and alpha.
5. Why can two runs using the same fixed h still diverge? Give two causes other than h.

LESSON 24.2 / MODULE 24

Entities, components, handles and data-oriented storage

> **Learning outcome:** Design identity separately from memory location.

Identity must survive storage changes

An entity is an identity in the simulated world. Its memory address is one possible temporary location, not its durable name. A vector can reallocate, a dense component store can move its last element into a removed slot, and an allocator can reuse a freed address for a different object. A pointer that still has plausible bits may therefore refer to the wrong lifetime.

Use a handle consisting of slot index and generation. A live slot stores its current generation; lookup succeeds only if the index is in range, the slot is alive and generations match. On destruction, invalidate the slot and advance its generation before reuse. A stale handle then fails instead of silently referring to the replacement entity. A finite generation counter can wrap, so choose a sufficiently wide counter and a stated wrap policy, such as permanently retiring an exhausted slot. [R11]

Components organize data around access

Begin with position and velocity components. An array-of-structures representation groups one entity's fields; a structure-of-arrays representation groups the same field across entities. The best choice depends on the hot queries and mutation patterns. Updating every position from every velocity may favor contiguous field arrays, but frequent heterogeneous operations can change the tradeoff. Measure a relevant workload rather than adopting data-oriented terminology as proof of speed.

A sparse-set design commonly maps entity slots to dense component indices and maps dense indices back to entities. Removal can swap the last dense element into a hole and update the moved entity's reverse mapping. This is efficient but makes borrowed references short-lived. A handle lookup at a synchronization boundary is safer than keeping an unchecked component pointer across structural changes.

Iteration and mutation need a transaction boundary

Destroying or creating entities while traversing a dense store can invalidate iteration and skip or repeat items. Queue structural commands during the update, then apply them at a defined boundary. Settle conflicting commands deterministically: a queued destroy followed by a write to the same handle should reject or follow a documented policy. Do not let vector iteration order determine the outcome accidentally.

Deferred destruction does not mean an entity is safe for all uses until memory is freed. The world must define whether a pending-destroy entity participates in later systems within the same tick. GPU resource retirement is another layer: removing an entity does not free a mesh still referenced by in-flight draws.

Durable IDs and runtime handles are different

A save file may use stable scene IDs and translate them to runtime handles when loaded. Saving only slot index and generation couples the format to a transient allocation history. Preserve relationships using the durable identity scheme, then resolve them transactionally after creating the new world.

The required evidence includes stale-handle tests, duplicate destruction, invalid index, slot reuse and mutation during iteration. Benchmarks should include creation, destruction and common queries, with input

size and iteration order stated. A faster store that accepts stale handles has failed the problem it was meant to solve.

Worked example

Handle (7,3) references slot 7 at generation 3. Destroy the entity, increment that slot to generation 4 and reuse it. Lookup of (7,3) must fail; lookup of the new (7,4) succeeds. Comparing only index 7 would confuse two different lifetimes.

In a dense position array, entity A is at index 0, B at 1 and C at 2. Remove B by moving C into index 1. Update C's sparse mapping to 1, shorten the dense array, and invalidate B's mapping. A borrowed pointer to index 1 now names C, demonstrating why address-based identity is unsafe.

During a tick, queue destroy(B) while iterating A,B,C. Complete the selected iteration policy, then apply the command once at the barrier. Test that C was not accidentally skipped by an immediate erase and that a repeated destroy(B) is rejected or treated idempotently according to the documented API.

Original lab assignment

Implement a small generational entity store with two components and deferred destruction. Pass: stale handles fail safely, iteration survives queued changes, and benchmarks include creation, deletion and common queries.

Assigned study

R20: game-state organization; R26: gameplay architecture for comparison; R11: lifetime and interface guidance.

Focused study: R11: lifetime, ownership and interfaces. R20: game-state organization. R26: compare, rather than replace, the small implementation.

Check your understanding

1. Why separate identity from an object address?
2. Is handle (7,3) valid for a slot now at generation 4?
3. Why defer structural changes during a component iteration?
4. After swap-removing dense element B and moving C into its slot, which mapping must change?
5. Why should a durable scene ID normally be separate from a runtime slot/generation handle?

LESSON 24.3 / MODULE 24

Allocators, task graphs and frame-resource lifetime

> **Learning outcome:** Make temporary storage and parallel dependencies observable.

Temporary memory is a lifetime claim

A linear arena allocates by advancing an offset through a fixed block. Resetting the offset is cheap only because the program promises that no remaining consumer will use any old allocation. That promise includes worker tasks, callbacks and GPU work, not just the function that allocated the memory. An arena's convenience does not exempt it from alignment, bounds or object-lifetime rules.

For size n and alignment a, round the current offset up to an aligned position before checking that n fits. Use overflow-safe arithmetic. A C++ byte arena that returns raw storage does not automatically construct objects or call destructors. The small exercise allocates byte ranges only; extending it to nontrivial C++ objects requires explicit lifetime management. [R11]

A task graph exposes ordering

A task is work with declared inputs, outputs and dependencies. A directed acyclic graph permits a task only after its predecessors have produced the required outputs. First implement a serial topological executor with cycle detection and stable tie order. This provides a reference execution order and makes missing dependencies easier to reproduce before adding scheduling nondeterminism.

Edges alone are not a complete race proof. If two tasks write the same array but neither depends on the other, a graph scheduler may legally run them together while the program races. Declare resource access or partition ownership so the graph can be checked against the actual data. Separate a logical task graph from the scheduling strategy that maps ready tasks to worker threads.

The critical path bounds improvement

If A and B can run independently and C consumes both, ideal parallel time is at least max(time(A),time(B)) +time(C), not the sum divided by worker count. More workers do not shorten a serial dependency chain. Scheduling overhead, cache behavior, contention and load imbalance can increase actual time beyond that lower bound.

Use a trace with start, finish, worker, dependencies and named resources. A bar chart is less useful when tasks are labeled only job1 and job2. Compare a serial run and a parallel run on identical inputs and validate results before analyzing speedup. A parallel task graph that produces wrong output has no meaningful performance win.

Frame ownership joins CPU and GPU lifetimes

A frame arena can contain CPU draw packets, while a staging arena may contain bytes needed for GPU upload. Those may have different retirement points. The CPU packet arena can reset after all CPU consumers copy what they need; GPU-visible staging storage may require submission completion. Grouping both under the name frame memory does not make their lifetimes equal.

Use a live-use token or completion serial in the reference model. Refuse reset while any use is outstanding. In a multiqueue or multithreaded engine, the token must actually represent every consumer. A counter modified without synchronization merely replaces one lifetime bug with a race.

Make failure tests constructive

Submit a task that retains a range, then attempt reset; the checked model must reject it. Add a dependency cycle and verify an actionable error. Remove a required edge in a scratch copy and demonstrate the discrepancy with the serial oracle. Record the limitation that the packaged scheduler model is serial; a race-free parallel executor is part of the full lab, not proven by a topological-sort test.

Worked example

An arena has capacity 64 bytes and offset 17. A request for 8 bytes aligned to 16 begins at 32 and ends at 40. The padding is 15 bytes, and the next offset is 40. A request that would end above 64 must fail without changing the previous offset.

Tasks A=2 ms and B=5 ms are independent; C=3 ms requires both. Serial time is 10 ms. Ideal unlimited-worker time is at least 5+3=8 ms, for a maximum speedup of 1.25 under this model. Scheduling overhead can make the actual speedup smaller.

If a frame arena's last use is submission 42 and the completed serial is 41, resetting it is invalid even when the CPU has already recorded frame 43. Its lifecycle follows the consumer's completion, not the producer's frame number.

Original lab assignment

Create a serial-then-parallel task graph and a checked temporary arena. Pass: resetting live storage is detected in tests, dependency order is verified, and the critical path is identified from a trace.

Assigned study

R11: ownership and concurrency; R20: engine architecture; R26: engine systems as a comparative implementation.

Focused study: R11: ownership and concurrency. R20: engine architecture. S05/S07: extend the ownership proof through GPU completion.

Check your understanding

1. When is resetting an arena safe?
2. Can task C consume A before A has completed its required output?
3. Why implement a serial graph executor first?
4. At offset 17, what range is used by an 8-byte allocation aligned to 16, and how much padding is inserted?
5. For independent A=2 ms, B=5 ms and C=3 ms after both, what are serial time and ideal parallel lower bound?

LESSON 24.4 / MODULE 24

Subsystem boundaries, save formats and reloadability

> **Learning outcome:** Choose interfaces that make change and failure manageable.

Boundaries should reduce the cost of change

Separate world rules from rendering, platform I/O and asset loading. A headless world update should not need a window or a graphics device. The renderer consumes a snapshot or draw description, not unrestricted ownership of every mutable game object. The resource manager resolves asset identities to runtime resources under a declared lifetime policy. The editor requests validated changes through a command interface rather than directly corrupting internal arrays.

An interface should say who owns inputs, how long borrowed data remains valid, which thread may call it and how failure is reported. A function name like load is not enough: does it block, return a ticket, mutate the live world or build a separate candidate? Choosing these semantics is more important than producing a large inheritance hierarchy. [R11]

Saves are a schema, not a memory dump

A durable save stores supported data under an explicit format version. Use stable object IDs, scalar values in declared units and references to named or hashed assets. Do not serialize pointers, container implementation bytes, device handles or accidental padding. Validate types, bounds, finite numbers and referential integrity before committing data to a world.

Migration converts a supported old schema into the current logical model. Unknown future versions should fail clearly unless a deliberate compatibility policy exists. Preserve the original file on a failed migration. A successful parse is not a successful load if it leaves half of the old world replaced by half of the new one.

Load transactionally

Read into bounded temporary storage, validate the complete candidate, resolve its references and then publish it at a controlled boundary. The old world remains usable if any earlier step fails. For saving, write a temporary sibling, flush as required, then replace the destination according to the platform's documented behavior. Atomic replacement is not the same as guaranteed durability after power loss; durability requirements can need additional filesystem and directory synchronization.

A small JSON teaching schema is useful because it is inspectable, but JSON does not remove validation duties. Some parsers accept nonstandard NaN values; reject nonfinite transforms explicitly. Limit file bytes, entity counts, strings and nesting appropriate to the declared threat model. A toy parser is not a sandbox for hostile arbitrary files.

Reload creates overlapping generations

When a shader or texture changes, create a candidate resource while the old resource remains active. Validate the candidate, switch the public handle at a safe boundary and retire the old generation only after its users finish. A failed candidate should leave the old working resource intact. Watch for stale asynchronous completions overwriting a newer successful version; associate requests with a generation or revision token.

Shutdown tests the architecture

Initialize subsystems in an order that matches dependencies and clean up in reverse ownership order. Inject failure after platform initialization, after device creation and after partial asset creation. RAII and scoped ownership help only when wrappers encode the real asynchronous lifetime. A destructor that frees GPU-visible memory too soon is still wrong even though it runs automatically.

The full lab requires one version migration, one malformed-load preservation test, one reloadable asset type and partial-initialization cleanup. The packaged Python schema helper demonstrates validation and migration; it is not a claim that a complete engine's shutdown has already been verified.

Worked example

Version 1 stores an entity's position as x,y,z fields. Version 2 stores position as a three-number array and adds a stable string id. A migration must define where that ID comes from; the teaching fixture uses a known legacy id field and refuses data without it rather than inventing unstable identities.

Load a candidate containing position=[1,2,NaN]. A permissive JSON parser may produce a floating NaN, but schema validation rejects it. The existing world must retain its prior position. Testing only that an exception occurs is insufficient; assert the old world is unchanged.

A texture reload request for revision 5 finishes after revision 6 has already been published. Compare the completion's revision with the current request token. Discard the stale revision-5 result and retire its temporary allocation; never overwrite revision 6 merely because a callback happened later.

Original lab assignment

Add versioned saves and one reloadable asset type. Pass: old supported data migrates, malformed data is rejected without partial world corruption, and subsystem shutdown works after injected initialization failure.

Assigned study

R20: resource management and tools; R26: serialization and gameplay architecture; R11: interfaces and error handling.

Focused study: R11: interfaces and ownership. R20/R26: resources and serialization. Carry Module 20 transactional and failure-testing practices into the engine.

Check your understanding

1. Why is a raw memory pointer unsuitable for a durable save?
2. What must a format version enable?
3. Why test partial initialization failure?
4. Why does atomic file replacement not automatically prove durability after sudden power loss?
5. Revision 5 completes after revision 6 is active. What should the reload system do and why?

Module 24 review

1. What does dropping accumulated ticks sacrifice?
2. Why is a valid handle lookup not permission to hold a component pointer forever?
3. Can an acyclic task graph still race?
4. What makes a load transactional?
5. What protects against an old reload overwriting a new one?

Project handoff

Retain the headless engine core evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Module 25 — Asset pipelines, editors and an integrated engine

Prerequisites: Module 24 and the software-engineering practices of Module 20.

Planning allocation: 70-110 planning hours, including the project's work and review.

Lesson	Outcome
25.1 — Importing, cooking and content-addressed asset builds	Treat assets as compiled inputs with dependency tracking.
25.2 — Streaming, residency and memory budgets	Load content without allowing asynchronous work to violate ownership.
25.3 — Editors, inspection and undoable commands	Make engine state understandable and safely editable.
25.4 — Engine integration and reproducible demonstration	Deliver a small complete system whose limitations are measurable.

Original module project

Gate E: deliver a small 3D engine demonstration with an asset cooker, reproducible scene, inspector, save/load, diagnostics and a measured frame/memory budget. A second person must build and run it from the README.

Integrating milestones

1. P25-A: reproducible mesh/image cooking with dependency-aware keys.
2. P25-B: bounded loading, cancellation and peak-budget evidence.
3. P25-C: validated inspector commands and undo/redo histories.
4. P25-D: clean build, measurements, second-person review and Gate E defense.

Evidence to retain

- Source/cooked manifest and provenance; cold/warm/change/corruption tests.
- Peak residency ledger; ten-command history tests; raw timings; reviewer record; explicit limitations.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 25.1 / MODULE 25

Importing, cooking and content-addressed asset builds

> **Learning outcome:** Treat assets as compiled inputs with dependency tracking.

Treat content as compiled input

Source assets are convenient for authors; runtime assets are convenient for loading, validation and execution. An importer interprets a source representation. A cooker converts it into the engine's declared runtime representation. A build system decides whether the conversion is necessary and records dependencies. Keeping these roles separate makes a failed asset build understandable and prevents the shipping runtime from depending on every authoring tool. [R20, R26]

Start with a deliberately small source schema: an authored triangle mesh and a tiny RGB image. Validate counts, finite coordinates, channel ranges and schema versions before allocating from untrusted sizes. Declare supported units, coordinate conventions, vertex layout and color semantics. Supporting one representation correctly is more useful than accepting an arbitrary filename and crashing later.

A cache key must describe every relevant input

A content-addressed build key should cover source bytes or a canonical source representation, dependency digests, importer/cooker version, target format and settings. A change to compression, color treatment or coordinate conversion changes the output contract and must invalidate the old result. A timestamp alone is not a trustworthy content identity.

Use an unambiguous encoding for the key description. Raw concatenation can collide structurally: bytes ab followed by c are indistinguishable from a followed by bc unless lengths or structure are encoded. The teaching cooker hashes a canonical JSON description containing named fields and dependency digests. Sorted keys and fixed separators remove irrelevant serialization differences in that description; they do not make every floating-point or source-format conversion universally deterministic.

Dependencies make rebuilds transitive

If a material depends on a texture and the texture changes, the material's build decision must see the updated dependency digest. A mesh may depend on conversion settings or referenced material definitions even when vertex bytes stay identical. Write the dependency graph, reject cycles where unsupported and store a manifest identifying exactly which inputs produced each output.

Separate reproducibility from portability. Identical bytes under the declared cooker/toolchain are a testable target. Different compilers or compression libraries may produce different but valid outputs. Include versions in the record and do not promise bit-identical cross-platform results without evidence.

Publish only complete artifacts

Build into a temporary location, validate the output, then publish it. Keep existing valid artifacts intact when a source is corrupted. A cache hit should verify the expected artifact digest or other appropriate integrity check; the existence of a filename is not proof that its contents are complete. Keep diagnostic messages tied to source path and field or element index.

The included Python cooker uses the standard library and a documented small format, not glTF or a production image compressor. It emits a triangle stream for the C++ CPU renderer and a linear-RGB image mip payload, plus a manifest. Its source image and mesh are original teaching fixtures with a provenance record. The image payload demonstrates cooking and can be inspected by tests; integrating that payload into the runtime texture sampler is a learner task.

Measure the cold and warm paths

Run a clean cook, then an unchanged cook, then a setting change, then a dependency change. Record which keys and outputs change. A malformed-source test must leave a readable error, not a half-valid runtime asset. This is the same engineering discipline used for compiling source code, now applied to the engine's content.

Worked example

Let the key description contain source_sha256, dependency_sha256, cooker_version='ge6-1', target='mesh-f32le-v1' and settings={scale:1.0}. Changing scale to 0.01 changes the key even if the source bytes are identical. Changing an absolute checkout directory should not change it because that path is not a semantic input.

The teaching GEM1 mesh format starts with four magic bytes, a little-endian uint32 vertex count, then eight little-endian f32 values per vertex: x,y,z,u,v,r,g,b. Six vertices use 8+6*32=200 bytes. The loader rejects a count not divisible by three, truncated bytes, extra unexpected payload and nonfinite fields.

An unchanged build in two fresh output directories should produce the same artifact bytes and manifest under the tested environment. That demonstrates the declared fixture's reproducibility; it does not prove that every future importer will behave identically.

Original lab assignment

Build a cooker for one mesh and one image representation. Pass: identical declared inputs reproduce the same result, changed settings invalidate the cache, and corrupted sources produce readable failures.

Assigned study

R20: asset pipelines; R26: asset management and content tools. Apply Module 20 dependency and provenance practices.

Focused study: R20/R26: asset pipeline roles. S03: compare a real interchange schema to the small teaching schema. Reuse Module 20 provenance and dependency practices.

Check your understanding

1. Why distinguish source assets from runtime assets?
2. Must a compression-setting change invalidate a cached build?
3. Why include tool version in a cache key?
4. Why is hashing raw concatenation of variable-length input blobs risky without framing?
5. How many bytes does a GEM1 asset with six vertices occupy under the stated eight-f32-per-vertex layout?

LESSON 25.2 / MODULE 25

Streaming, residency and memory budgets

> **Learning outcome:** Load content without allowing asynchronous work to violate ownership.

Loading is a state machine with ownership

An asynchronous request should move through explicit states such as requested, reading, decoding, waiting-for-upload, uploading, resident, retiring, cancelled and failed. Each state has an owner and a permitted resource set. The renderer uses a stable placeholder until the required representation is resident. A file read completing does not mean a usable GPU texture exists. [R20, R26]

Separate the request identity from the resource identity. A handle names the desired asset; a ticket identifies this particular load attempt. Reload, scene change or cancellation can make a completion stale. Check a generation or revision token before publishing the result. Late completions must release their temporary allocations without reviving an unwanted asset.

Bound every expensive stage

A small compressed file can expand into a much larger decoded image. During transition, memory can include compressed input, decoded CPU bytes, staging/upload storage, new GPU resident bytes and an old generation waiting for retirement. Count peak overlap, not merely steady-state resident payload.

Maintain separate budgets when memory pools are separate, such as CPU decode and GPU device-local memory. A combined budget model is useful for a simplified exercise but must not be mislabeled as actual hardware accounting. On unified-memory systems, logical CPU/GPU payload counters can double-count shared allocations unless their ownership is understood. Record the estimator's assumptions.

Limit queue depth and in-flight decoded bytes, not just the number of worker threads. Two workers each holding several enormous decoded assets can exceed a memory budget despite a small thread pool. Admission should reserve the expected peak cost before beginning a stage or choose a fallback when an estimate is unavailable.

Cancellation is not magic reclamation

A cancellation flag can prevent future stages or publication, but it may not interrupt an operating-system read or a submitted GPU upload. Keep the allocation until the active consumer finishes, then retire it. Never free upload storage simply because the user changed scenes. A state machine should distinguish cancelled-but-in-flight from fully reclaimed.

Eviction also has conditions. An asset may be logically unused by the world but still referenced by a submitted frame. Use reference or pin counts for current owners and completion tracking for asynchronous users. Avoid evicting a fallback asset that every missing texture relies on.

Budget failure is an ordinary case

Force a tiny budget so the test can reliably exercise rejection and fallback. Select lower-resolution content, defer an optional request, evict eligible content or retain the placeholder according to an explicit priority policy. Essential content can fail the scene load with an actionable error. An endless retry loop that repeatedly decodes an impossible asset is not graceful degradation.

Record request count, rejected admissions, pending bytes, peak decoded bytes, resident bytes and retirement backlog. Add a cancellation test immediately before completion. The teaching budget and state

models help reason about these cases; the complete multithreaded streaming subsystem and GPU-residency measurement remain lab work.

Worked example

Assume an explicitly combined teaching budget of 100 MiB. The engine holds 40 MiB resident data and 20 MiB of existing temporary/retiring data. A new request needs 30 MiB decoded staging plus 20 MiB of new resident data at peak. Its estimated peak is 40+20+30+20=110 MiB, so admission fails even though final resident data would be only 60 MiB after cleanup.

The valid response may be to finish retiring the 20-MiB old allocation, then admit the request at a 90-MiB peak. It is invalid to free that old allocation early while an in-flight command still references it.

Request ticket 12 is cancelled while upload is submitted. Completion can move it to retired, releasing staging after the consumer finishes, but cannot publish it as resident. A newer request ticket 13 for the same asset is a separate lifetime and must not inherit ticket 12's cancellation or completion accidentally.

Original lab assignment

Implement a bounded streaming queue with placeholder assets and cancellation. Pass: a forced small budget produces graceful fallback, no use-after-free, and a report of peak rather than only steady-state memory.

Assigned study

R20: resource loading; R26: asset management, asynchronous loading and streaming concepts.

Focused study: R20/R26: asynchronous loading and streaming. S05/S07: retirement follows actual consumers. Keep the memory estimator distinct from device measurement.

Check your understanding

1. Why is compressed file size not the complete memory budget?
2. What should rendering do while an optional asset is unavailable?
3. Why model loading as explicit states?
4. Under the example 100-MiB budget, why is a request ending at 60 MiB resident still rejected?
5. Does cancelling an already submitted upload make its staging allocation immediately reusable?

LESSON 25.3 / MODULE 25

Editors, inspection and undoable commands

> **Learning outcome:** Make engine state understandable and safely editable.

An inspector is a controlled view of the model

An editor makes engine state inspectable without becoming a second inconsistent copy of the world. The model owns authoritative entity and asset data. The view presents that data and captures user intent. A controller or command layer validates changes, applies them at a safe boundary and reports failure. A text inspector is acceptable for this small engine; correctness matters more than widget count. [R20, R26]

Show stable identity, transform values, component presence and relevant resource status. Distinguish authoring state from transient simulation state. Editing a simulated position may update an authoring transform, teleport a live body or be disallowed while playing; choose and label the behavior. Do not silently save transient physics output as the source scene.

Commands make changes reversible

Represent a committed transform edit as a command containing entity identity, before value and after value. Apply validates the current target and new value; undo restores the before value under the appropriate conflict policy. Redo reapplies the after value. A new edit after an undo invalidates the redo branch unless the editor deliberately supports branching history.

A mouse drag can emit hundreds of previews but should usually commit one meaningful command. Save the initial value when the drag begins, update a temporary preview while dragging, and push a single before/after command on commit. Cancel discards the preview. This distinction keeps undo useful and prevents partially edited state from becoming a durable save.

Validation belongs below the UI

Reject nonfinite numbers, missing entities, invalid scale ranges under the chosen model and unsupported operations. A text box can be bypassed by a script or file import; validate at the command/model boundary as well. A negative scale may be valid if the engine handles reflected winding and tangent frames, but it should not be accidentally accepted by a renderer that assumes only positive scales.

Undo is a state transition, not an exemption from validation. The target entity may have been deleted or reloaded. Define how stable IDs resolve and how incompatible history is cleared. In this teaching engine, a scene load clears history because commands refer to the previous scene revision. A richer editor may instead store explicit revision-aware commands.

Input focus prevents unintended actions

Keyboard input intended for a numeric field must not simultaneously move the camera or trigger gameplay. Route events through a declared focus/capture policy. Pressing a gameplay key while a field has focus should edit or be ignored according to the UI, not issue an unrelated world command. Losing focus during a drag needs a commit or cancel policy.

Test histories, not only buttons

Create ten transform commands, undo all ten, redo all ten and compare exact logical state. Then undo three and make a new edit; verify redo is empty. Attempt an invalid command and ensure neither world state nor

history changes. Save, reload and compare committed values. The supplied command-history helper demonstrates the data model; the full inspector, focus integration and engine coupling remain required practical work.

Worked example

Entity lamp has position (0,0,0). A committed move stores before=(0,0,0), after=(2,0,0). Undo restores the former; redo restores the latter. Three previews at x=0.5,1.0,2.0 during one drag still produce just one command when committed.

After commands A,B,C, undo C and B. The model is at A and the redo stack can restore B then C. Commit new command D now: the correct ordinary linear history is A,D, and the redo branch containing B,C is cleared. Keeping it would allow replaying commands whose assumed before-state is no longer the current history.

A request to set x to NaN is rejected before mutation. Verify both the entity transform and the lengths of undo/redo history are unchanged. The absence of a crash is not sufficient; the command must be transactionally rejected.

Original lab assignment

Create a scene inspector with transform editing and a ten-command undo/redo test. Pass: invalid inputs are rejected, focus does not trigger unintended gameplay, and save/reload preserves committed changes.

Assigned study

R20: game tools; R26: editor and reflection concepts. Use a minimal UI toolkit appropriate to the selected platform.

Focused study: R20/R26: tools and model inspection. R11: command interface contracts. Apply Module 20 input-validation and transactional-state habits.

Check your understanding

1. What is the model/view distinction useful for?
2. What data can make a transform move reversible?
3. Why distinguish preview state from committed authoring state?
4. After undoing two commands, what should a new edit do to an ordinary linear redo stack?
5. Why should a drag preview not normally be written directly into the saved authoring scene before commit?

LESSON 25.4 / MODULE 25

Engine integration and reproducible demonstration

> **Learning outcome:** Deliver a small complete system whose limitations are measurable.

Integration proves the boundaries agree

Gate E is a small complete engine demonstration, not a collection of disconnected attractive screenshots. Its CPU reference renderer, selected GPU route where available, fixed-step world, entity/resource ownership, asset cooker, inspector, versioned saves and diagnostics must agree about units, identity, coordinates and lifetime. The project can be visually simple. A few meshes, a controlled light and a moving entity are enough to expose real architecture problems.

Preserve a headless world path so tests run without a display. Keep a fixed reference scene with original or properly attributed assets, a deterministic input sequence under the declared reproducibility scope, and a saved camera. Build and cook commands should work from a clean extracted tree, not depend on an absolute path or a file that exists only in a developer's cache.

Budget against a declared machine

Choose resolution, quality settings, scene contents and a test environment. A 60-fps target corresponds to about 16.667 ms per presented frame, but it is an example budget, not a universal passing score. A CPU reference renderer may have a much slower target and still provide valuable correctness evidence. State separate budgets for CPU simulation/recording, GPU execution when measured, full-frame pacing and memory.

Record warm-up, sample count, timer scopes and raw samples. Report median, p95, p99 and maximum for the selected workload, using a stated percentile definition. Never relabel a CPU timer around submission as GPU time. API timestamp queries require their own support, units and synchronization. Separate initialization and asset-cook time from steady-state frame time.

Use adversarial integration scenes

Test a triangle crossing the near plane, rapid resize/minimize/restore, stale handles, malformed assets, cancelled loads, save migration, invalid editor input and shutdown with pending work. Failure injection should have a predictable expected state. An application that survives by silently disabling half its features needs that quality change in the report.

Image comparisons require a known reference, matching camera, extent, sample convention and color domain. The supplied CPU renderer's integer-quantized coverage can differ from hardware at edges. Compare interiors with an appropriate tolerance and separately inspect edges; do not use a broad tolerance to hide a flipped image or incorrect UV interpolation.

Reproducibility needs another set of hands

The canonical checkpoint requires a second person to build and run the engine from its README. A clean-directory automated test is useful but not the same evidence. Give the reviewer the source tree and commands, record what they did, and fix every undocumented assumption. Ask them to reproduce one boundary test and one live edit, then inspect a save/reload result.

The learner must explain one CPU/GPU dependency and one quality/performance tradeoff. A reference implementation copied without understanding does not pass the oral review. Existing engines may be used for comparison, but cannot replace the central implementation and still meet this checkpoint's objective.

Close with a limitations record

Mark capabilities implemented, tested, pending or deliberately out of scope. The bundle itself includes educational components and reference tests; it does not certify the learner's Gate E engine or claim a hardware run not performed. A trustworthy limitations section makes the next volume possible: Game Systems and Production builds on the engine's actual contracts, not its aspirational feature list.

Worked example

At 60 frames per second, the ideal interval is 1000/60=16.6667 ms. If 99 frames take 10 ms and one takes 200 ms, the mean is 11.9 ms, which sounds comfortably below the budget. Yet the 200-ms stall remains visible. Under nearest-rank percentiles, p95=10 ms, p99=10 ms and maximum=200 ms for this particular 100-sample set. This demonstrates why a maximum, hitch count and sufficient sample count matter even when percentiles are reported.

For ten sorted samples [8,8,9,9,10,10,11,12,18,40], nearest-rank p95 is the element at ceil(0.95*10)-1, which is index 9: 40 ms. State the definition so another implementation can reproduce the number.

A clean build in an empty directory can reveal a missing generated header or uncooked asset. A second-person build can additionally reveal an ambiguous instruction such as run the engine without saying which working directory or executable. Both tests have value, but they answer different questions.

Original lab assignment

Complete Gate E using the project specification in the handbook. Pass: another person follows the README, a headless test scene passes, all assets have provenance, and a limitations section identifies remaining correctness or performance risks.

Assigned study

R20: project completion; R26: profiling and engine architecture comparison; revisit Module 20 release and review practices.

Focused study: Canonical Gate E specification; R20/R26: engine integration and profiling. Revisit Module 20 clean-build, provenance and review requirements.

Check your understanding

1. What frame interval corresponds to 60 frames per second?
2. Why report more than average frame time?
3. What does a clean-machine build test reveal?
4. For 99 frames at 10 ms and one at 200 ms, compute the mean and explain why it hides a problem.
5. Why is a clean-directory automated build not sufficient to mark the second-person review complete?

Module 25 review

1. Which key inputs prevent a stale content build after a tool upgrade?
2. Why can bounded worker count still exceed the memory budget?
3. What must happen to editor history when its target scene revision is replaced?
4. What does the software-only Gate E allowance exclude?
5. What must a reviewer reproduce besides opening a screenshot?

Project handoff

Retain the gate e integrated engine evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Gate E specification

After Volume 5. Build an engine demonstration containing a CPU reference renderer, one supported GPU backend where available, a fixed-step simulation, explicit entity/resource ownership, an asset cooker, an inspector and versioned save/load. A software-only execution path is acceptable while GPU access is unavailable, but untested GPU claims must remain excluded.

Deliver a fixed scene, source assets with provenance, build/cook commands, validation logs, headless tests, screenshot or image-difference evidence, frame-time data and a peak-memory report. The budget is chosen for the declared test machine; 60 fps is an example target, not a universal pass requirement. Test near-plane crossing, resize/minimize, stale handles, malformed assets and resource shutdown.

A second person must build from a clean checkout or extracted source tree. The learner must explain one CPU/GPU dependency and one quality/performance tradeoff. Using an existing engine for comparison is encouraged; using it in place of the central engine implementation changes the learning objective and needs a separate implementation project.

Use the companion specification

CAPSTONE_SPEC.pdf expands this preserved checkpoint into milestones, acceptance criteria, failure-path tests, a measurement plan and an independent-review rubric. Carry the five module projects into one integrated system; do not count Gate E as an unrelated sixth project. Use templates/REVIEWER_SIGNOFF.md for the actual second-person build and demonstration.

The included code is reference material for selected contracts, not a finished engine submission. Keep implemented, tested, pending and excluded capabilities distinct. A software-only route may proceed while GPU access is unavailable, but no GPU correctness or performance claim passes without an actual run.

Sources and provenance

How this edition relates to the roadmap

Package 06 expands Volume 05 of the September 28, 2026 Computer Science and Computer Engineering for Games manifest. The source manifest and original curriculum were inspected directly. Module/lesson IDs, titles, learning outcomes, original lab assignments, original reading assignments and the first three lesson quiz questions are retained. The additional teaching, worked derivations, procedures, two application questions per lesson, module reviews and code in this bundle are original additions.

The canonical lesson questions were transcribed from the original curriculum PDF's Volume 05 section, not inferred from the shorter manifest. Original assessment examples are retained where a question refers to an example. Answer explanations are expanded. The complete original program and textbooks are not repackaged here.

Reading IDs R11 and R19-R27/R37 retain their original program meanings. S01-S16 are supplementary references for this volume only. Technical source links below were consulted or retrieved on September 28, 2026; S10 is specifically marked as a link-level check. API documentation can change. Record the version actually used in your own lab; this edition does not certify any unspecified browser, SDK, driver or engine release.

Canonical assigned references

R11 — C++ Core Guidelines editors: C++ Core Guidelines

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R19 — Carnegie Mellon University: 15-462/662 Computer Graphics

Rendering, geometry and animation course structure and exercises.

https://15462.courses.cs.cmu.edu/fall2021/

R20 — Carnegie Mellon University: 15-466/666 Computer Game Programming

Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

R21 — Khronos Group: Vulkan tutorial

Explicit graphics API study; feature-check the device before using the moving latest tutorial.

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

R22 — SDL project: SDL3 documentation

Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples.

https://wiki.libsdl.org/SDL3/FrontPage

R23 — Matt Pharr, Wenzel Jakob, Greg Humphreys: Physically Based Rendering, fourth edition

Rendering mathematics and implementation; use chapter titles in lesson assignments.

https://www.pbr-book.org/4ed/contents

R24 — Microsoft: Direct3D 12 graphics

Windows graphics backend reference.

https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics

R25 — Apple: Metal developer resources

Apple-platform graphics and compute reference.

https://developer.apple.com/metal/

R26 — Epic Games: Programming with C++ in Unreal Engine

Engine comparison after C++ foundations, not a beginner substitute.

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

R27 — Glenn Fiedler: Fix Your Timestep!

Original practitioner explanation of fixed-step simulation; read critically and implement your own lab.

https://gafferongames.com/post/fix_your_timestep/

R37 — NVIDIA: CUDA Programming Guide

GPU execution and memory; CUDA is an optional vendor-specific lab route.

https://docs.nvidia.com/cuda/cuda-programming-guide/

Supplemental primary sources

S01 — W3C: WebGPU Shading Language (WGSL)

Host-shareable layout, alignment, storage/uniform contracts; evolving specification.

https://www.w3.org/TR/WGSL/

S02 — Netpbm: PPM format specification

Strict PPM and the explicitly documented sRGB variant used in this package.

https://netpbm.sourceforge.net/doc/ppm.html

S03 — Khronos Group: glTF 2.0 specification

Material/color semantics and tangent handedness; glTF importing is assigned, not implemented here.

https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html

S04 — Pharr, Jakob and Humphreys: PBRT 4: Bounding Volume Hierarchies

Spatial bounds, construction and traversal.

https://www.pbr-book.org/4ed/Primitives_and_Intersection_Acceleration/Bounding_Volume_Hierarchies

S05 — Khronos Group: Vulkan Tutorial: Frames in flight

Frame contexts; distinguish frame indexing from presentation-image indexing.

https://docs.vulkan.org/tutorial/latest/03_Drawing_a_triangle/03_Drawing/03_Frames_in_flight.html

S06 — GPU for the Web Community Group: WebGPU explainer

Conceptual API overview; an explainer, not a substitute for the normative specification.

https://gpuweb.github.io/gpuweb/explainer/

S07 — Khronos Group: Vulkan Guide: Synchronization examples

Dependencies, stages, accesses and barriers.

https://docs.vulkan.org/guide/latest/synchronization_examples.html

S08 — Khronos Group: Vulkan Guide: Swapchain semaphore reuse

Presentation waits may outlive the graphics submission fence.

https://docs.vulkan.org/guide/latest/swapchain_semaphore_reuse.html

S09 — Microsoft: Using resource barriers in Direct3D 12

Resource state and hazard management; do not transliterate Vulkan flags.

https://learn.microsoft.com/en-us/windows/win32/direct3d12/using-resource-barriers-to-synchronize-resource-states-in-direct3d-12

S10 — Apple: Synchronizing CPU and GPU work

Official Metal starting point. URL retrieved; dynamic page body was not fully inspectable during authoring.

https://developer.apple.com/documentation/metal/synchronizing-cpu-and-gpu-work

S11 — SDL: SDL_GetWindowSizeInPixels

Drawable pixel extent can differ from logical window size.

https://wiki.libsdl.org/SDL3/SDL_GetWindowSizeInPixels

S12 — Khronos Group: Vulkan Tutorial: Swap chain recreation

Out-of-date/zero-extent paths and safe fence-reset sequencing.

https://docs.vulkan.org/tutorial/latest/03_Drawing_a_triangle/04_Swap_chain_recreation.html

S13 — Pharr, Jakob and Humphreys: PBRT 4: Radiometry

Radiometric quantities and their domains.

https://www.pbr-book.org/4ed/Radiometry,_Spectra,_and_Color/Radiometry

S14 — Pharr, Jakob and Humphreys: PBRT 4: Roughness using microfacet theory

Microfacet distributions and model assumptions.

https://www.pbr-book.org/4ed/Reflection_Models/Roughness_Using_Microfacet_Theory

S15 — Pharr, Jakob and Humphreys: PBRT 4: Image texture

Texture reconstruction, footprints and filtering.

https://www.pbr-book.org/4ed/Textures_and_Materials/Image_Texture

S16 — GPU for the Web Community Group: WebGPU samples

Optional companion examples; no external sample source is bundled.

https://webgpu.github.io/webgpu-samples/

Scope of evidence

The diagrams in this edition are numerical reference-renderer outputs, not external photographs or a claim of hardware-rendered results. All included mesh, image and world fixtures are original procedural teaching data; assets/source/PROVENANCE.md documents them. Code is original and standard-library based except for the platform APIs explicitly used in the optional browser bridge. No textbook chapters, proprietary SDK binaries, third-party game assets or font files are redistributed.

The small examples illustrate contracts, not all details of their parent fields. PBRT supplies deeper transport/acceleration study; platform documentation supplies exact native API rules. The course's broader undergraduate-to-graduate pathway remains self-study, not accreditation, a degree, or a guarantee of employment.